

UNIDAD 2

Frameworks para escalar SCRUM

Dado que Scrum impone un límite en la cantidad máxima de integrantes del Equipo de Desarrollo, surge la necesidad de escalar este Framework para poder gestionar productos de mayor envergadura, ya que las prácticas ágiles se utilizan en ambientes complejos para reducir tal complejidad. Existen diferentes frameworks para escalar como Nexus, scale scrum, less, SAFe.

Framework Nexus

Nexus es un Framework que consiste en roles, eventos, artefactos y técnicas que vinculan el trabajo de aproximadamente tres a nueve equipos Scrum que trabaja sobre un único Product Backlog común para la construcción de un incremento integrado de producto “Terminado”. Con un solo Product Owner para entregar un único producto.

Nexus surge para lidiar con la complejidad que supone varios Equipos Scrum trabajando sobre un mismo Product Backlog. Esta complejidad está dada por las siguientes dependencias entre equipos:

1. Requerimientos: el alcance de estos puede superponerse. La forma en que se implementan también puede afectar a los demás.
2. Conocimiento del dominio: el conocimiento del sistema de negocio debería mapearse a los Equipos Scrum para minimizar las interrupciones entre los mismos durante el Sprint.

Responsabilidades

- Nexus Integration Team: Responsable de asegurar que se produzca un incremento integrado al menos una vez en cada sprint. La integración incluye abordar las restricciones técnicas y no técnicas del equipo multifuncional que pueden impedir la capacidad de un Nexus para entregar constantemente un incremento integrado.
- Product Owner: Es responsable de maximizar el valor del producto y el trabajo realizado e integrado por los Scrum Teams en un Nexus, así mismo también es responsable de la gestión eficaz del Product Backlog.
- Scrum Master: responsable de asegurar que el marco de trabajo Nexus se entienda y se promulgue como se describe en la Guía de Nexus. Puede ser un Scrum Master en uno o más de los scrums teams en el Nexus.
- Uno o más miembros del Nexus Integration Team: a menudo está formado por miembros de los Scrum Teams que ayudan a los Scrum Teams a adoptar herramientas y prácticas que contribuyen a mejorar la capacidad de los Scrum Teams para entregar un Integrated Increment útil y de valor que cumple con la Definición de Terminado.

Eventos

Nexus agrega o extiende los eventos definidos por Scrum. La duración de los eventos Nexus se guía por la duración de los eventos correspondientes en la Guía de Scrum. Tienen definido un bloque de tiempo adicional a sus correspondientes eventos de Scrum.

- Nexus Sprint: Los scrums teams producen un solo Integrated Increment (Incremento de integración), es lo mismo que scrum.
- Refinamiento entre equipos: El Refinamiento Entre Equipos del trabajo del Product Backlog reduce o elimina las dependencias entre equipos dentro de un Nexus. El Product Backlog debe descomponerse para que las dependencias sean transparentes, se identifiquen entre equipos y se eliminen o se minimicen.

El Refinamiento Entre Equipos del Product Backlog a escala sirve a un propósito dual:

- Ayuda a los Scrum Teams a prever qué equipo entregará qué elementos del Product Backlog.
- Identifica dependencias entre estos equipos.

El Refinamiento Entre Equipos es continuo. La frecuencia, duración y participación del Refinamiento Entre Equipos varía para optimizar estos dos propósitos.

- **Nexus Sprint Planning:** El propósito de la Nexus Sprint Planning es coordinar las actividades de todos los Scrum Teams dentro de un Nexus para un solo Sprint. Los representantes apropiados de cada Scrum Team y el Product Owner se reúnen para planificar el Sprint. Resultados:
 - Objetivo de Sprint para cada Scrum team.
 - Objetivo de Nexus.
 - Un Nexus sprint backlog.
 - Un Sprint backlog.
- **Nexus Daily Scrum:**
 - Se discuten problemas de integración y se inspecciona el progreso hacia el objetivo de sprint del nexus.
 - Solo asisten representantes de cada Scrum team.
 - La Daily Scrum de cada Scrum Team complementa la Nexus Daily Scrum creando planes para el día, centrados principalmente en abordar los problemas de integración planteados durante la Nexus Daily Scrum.
 - Los Scrum teams no solo se reúnen o coordinan en el Nexus daily scrum, sino que, durante el día, puede existir comunicación entre equipos con mayor detalle sobre adaptar o replanificar el resto del trabajo del sprint.
- **Nexus Sprint Review:** La Nexus Sprint Review se lleva a cabo al final del Sprint para brindar retroalimentación al Integrated Increment terminado que el Nexus ha construido durante el Sprint y determinar adaptaciones futuras. La nexus sprint review reemplaza a las sprint reviews de cada scrum team.
- **Nexus Sprint Retrospective:** El propósito de la Nexus Sprint Retrospective es planificar formas de mejorar la calidad y la eficacia en todo el Nexus. El Nexus inspecciona cómo fue el último Sprint con respecto a individuos, equipos, interacciones, procesos, herramientas y su Definición de Terminado. Además de las mejoras de cada equipo, las Retrospectivas de los Scrum Teams complementan la Nexus Sprint Retrospective mediante el uso de inteligencia de abajo hacia arriba para centrarse en los problemas que afectan al Nexus en su conjunto.

Artefactos

Representan trabajo o valor y están diseñados para maximizar la transparencia. El Nexus Integration Team trabaja con los Scrum Teams dentro de un Nexus para garantizar que se logre la transparencia en todos los artefactos y que el estado del Integrated Increment se entienda.

Cada artefacto contiene un compromiso, y estos existen para reforzar el empirismo y el valor de Scrum para el Nexus y sus interesados.

- **Product Backlog:** Hay uno solo que contiene la lista de todo lo que el Nexus y sus Scrum Teams necesitan para mejorar el producto. El Product Backlog debe entenderse con tal nivel que permita detectar y minimizar las dependencias. Es responsabilidad del Product Owner. El compromiso es el Objetivo del Producto, que describe el estado futuro del producto y sirve como un objetivo a largo plazo para el Nexus.
- **Nexus Sprint Backlog:** está compuesto del Objetivo del Sprint de Nexus y elementos del Product Backlog de cada Scrum Team del Nexus. Se usa para resaltar las dependencias y el flujo de trabajo durante el Sprint, y se actualiza durante el mismo a medida que se obtiene más conocimiento. Este debe tener un nivel de detalle tal que permita que Nexus pueda inspeccionar su progreso en la Nexus

Daily Scrum. Su compromiso es el Objetivo de Sprint de Nexus, que es un único objetivo para Nexus. Es la suma de todo el trabajo y los Objetivos de Sprint de los Scrum Teams dentro del Nexus. Se crea en la Nexus Sprint Planning y se agrega al Sprint Backlog de Nexus. Los Scrum Teams lo tienen en cuenta a medida que trabajan. En la Nexus Sprint Review se debe de mostrar la funcionalidad de valor y útil que está terminada para alcanzar el Objetivo del Sprint.

- **Integrated Increment:** es la suma actual de todo el trabajo integrado completado por un Nexus en relación con el Objetivo del Producto. Se inspecciona en la Nexus Sprint Review pero puede entregarse antes de la misma. Debe cumplir con la definición de terminado. El compromiso de este artefacto es la Definición de Terminado, que define el estado del trabajo integrado cuando cumple con la calidad y las medidas requeridas para el producto. El incremento se termina sólo cuando está integrado, es de valor y utilizable. Es responsabilidad del Nexus Integration team una definición de terminado que se pueda aplicar en cada sprint, y todos los Scrum Team deben definir y adherirse a esta definición. Cada Scrum Team se autogestiona para llegar a este estado, pudiendo aplicar una definición de terminado más estricta pero nunca usar criterios menos rigurosos de lo acordado para el Integrated Increment.

FILOSOFIA LEAN

Es una filosofía o enfoque más viejo que agile, que nace en Japón con Toyota, y busca maximizar el valor generado al cliente con el mínimo consumo de recursos, argumentando que todo el esfuerzo que no cree valor, se considera desperdicio. Esto permite reducir los esfuerzos en crear artefactos que no agreguen valor a un producto, concentrando los esfuerzos en los aspectos esenciales.

La filosofía “Lean”, conduce a una visión integrada de la cultura y la estrategia para atender al cliente final con alta calidad, bajo costo y tiempo de entrega, produciendo exactamente lo que el cliente final quiere, cuando lo quiere, dónde lo quiere, a un costo mínimo y precio justo. Siendo el cliente quien determina si el servicio o producto que la empresa entrega tiene valor o no.

Para esto utiliza el método Just in Time, el cual permite el intercambio del producto entre fases del proceso de desarrollo o a la entrega del producto final al cliente justo a tiempo, esto es poco antes de que lo usen y en la cantidad justa y necesaria, “analice cuando lo necesite, no antes”.

Principios Lean

Estos principios apuntan al desarrollo de productos en la industria de manufactura. Como el software es una actividad de desarrollo de un producto intangible, a los principios originales de Lean, hubo que adaptarlos al desarrollo de software (matrimonio Poppendieck en “Lean SW Development”).

1. **Eliminar desperdicios:** Es quizás el más importante. Debemos tener un proceso que sea eficiente y que sume valor en todos los pasos del proceso, todo paso o parte que no contribuye a la generación de valor produce un desperdicio. Todo esto no quiere decir que el proceso es perfecto, porque según todas las filosofías de mejora continua (como esta o agile) marcan que siempre se puede mejorar el proceso.

Lo que se busca evitar:

- Producir funcionalidades en exceso, que luego no se usan (el 80% del valor está en el 20% de las funcionalidades [Pareto]).
- Retrabajo.
- Que los artefactos se vuelvan obsoletos antes de terminarlas.

Los 7 desperdicios en Lean: Lean busca eliminar actividades que consumen tiempo y esfuerzo, pero no aportan valor al producto.

- **Características extras**

Desarrollar funcionalidades o especificaciones innecesarias. Muchas veces los requisitos definidos demasiado temprano quedan desactualizados y generan trabajo inútil.

- **Trabajo a medias**

Tareas sin terminar (historias de usuario, bugs, casos de uso, etc.). Esto obliga a retomarlas después y genera desorden y multitarea.

- **Proceso extra**

Pasos o actividades dentro del desarrollo que no aportan valor al producto. Lean busca simplificar procesos y eliminar burocracia innecesaria.

- **Búsqueda de información**

Pérdida de tiempo buscando documentación, versiones o datos por falta de organización y trazabilidad. Se evita con una buena gestión de configuración (SCM).

- **Defectos**

Errores que pasan a etapas posteriores y generan retrabajo. La calidad y testing ayudan a prevenirlos desde el inicio.

- **Esperas**

Tiempo perdido esperando aprobaciones, respuestas o tareas de otros. Los equipos multidisciplinarios ayudan a reducir estas demoras.

- **Cambios de tareas (Multitasking)**

Pasar constantemente de una tarea a otra reduce la productividad por el tiempo de adaptación mental. Kanban limita el trabajo en progreso (WIP) para evitarlo.

2. **Amplificar el aprendizaje** : Tiene que ver con la transparencia y transformación de conocimiento implícito en explícito, lo cual fortalece al equipo de trabajo. Crear y mantener una cultura de mejoramiento continuo, donde los individuos intercambien sus experiencias y conocimientos para contribuir con el aprendizaje colectivo. Por otro lado, todo conocimiento que se genere debe compartirse para que sea fácilmente accedido por toda la organización.
3. **Embeber la integridad conceptual**; Se deben encastrar todas las partes del producto o servicio que tengan coherencia y consistencia (tiene q ver con los RNF) para hacer al producto más íntegro. Debemos pensar en la arquitectura desde el principio de la creación del producto → Construir calidad desde el principio, no probar después

Dos clases de inspecciones

- Inspecciones luego de que los defectos ocurren
- Inspecciones para prevenir defectos

¡Si se busca calidad no inspeccione después de los hechos! Si no es posible, inspeccione luego de pasos pequeños

Arquitectura pensamos en una estructura del producto → que sea escalable, robusto, con coherencia y consistencia entre sí y que sea mantenible en el tiempo.

4. **Diferir compromisos hasta el último momento responsable**:
 - Esto apunta a postergar la toma de decisiones lo suficientemente como para poder tener la mayor información necesaria para tomar esa decisión, pero a la vez, antes de que ese momento sea muy tarde (diferir la decisión irreversible).
 - Se debe diferir hasta el último momento responsable, ya que, si nunca tomamos la decisión, el no hacerlo también es una forma de decidir, porque otro lo va a hacer por nosotros (la vida, el cliente, otro), lo cual tiene consecuencias. Podemos caer en la parálisis del análisis.
 - Esto se puede reflejar en Agile con el Product Backlog, donde nunca se tiene el 100% de los requerimientos, sino que se va completando a medida que se tiene más información sobre lo que se necesita implementar. Las mejores estrategias de diseño de software están basadas en dejar abiertas opciones de forma tal que las decisiones irreversibles se tomen lo más tarde posible. Se relaciona con los principios ágiles de maximizar el trabajo no hecho y de recibir cambios de requerimientos aun en etapas finales.

5. **Dar poder al equipo:**

- Otorgar libertad de acción y poder de decisión al equipo. Para ello, el equipo debe ser multifuncional y autogestionado, y sus miembros deben estar capacitados y motivados.
- Esto implica no subordinar por parte de los superiores, debido a que esto anula las capacidades intelectuales, entrenar líderes, delegar decisiones y fomentar buena ética laboral. Pero a la vez, implica un compromiso por parte de los miembros del equipo, para desempeñarse y cumplir con lo pactado de forma completa y correcta. Este aspecto termina siendo un talón de Aquiles en las filosofías Agile y Lean, ya que en la práctica pocas veces se logran reunir todas estas características.
- Relacionado con el principio 11 del Manifiesto Ágil (“Las mejores arquitecturas, requisitos y diseños emergen de equipos autoorganizados”).

6. **Ver el todo:**

- En Lean se busca tener una visión holística, que permita asociar y comprender el todo, el producto, el valor agregado que hay detrás, el servicio que brinda el complemento de los productos, más allá de los objetivos particulares por áreas o gerencias, porque la idea es que los objetivos particulares de las partes estén alineados con los objetivos globales.

7. **Entregar lo antes posible:**

- La idea es entregarle al cliente software funcionando lo más rápido posible y que este producto sea útil para él. Se habla de entregarle software antes de que las necesidades de negocio cambien, porque el ambiente donde se desempeña el mismo cambia. Para esto Lean propone acotar ciclos de desarrollo (principio 1 de Agile: satisfacer al cliente con entregas tempranas) y entregar rápidamente incrementos pequeños de valor, lo que permite salir pronto al mercado con un producto mínimo que sea valioso (principio ágil de entregas tempranas y frecuentes de software de valor para el cliente).

KANBAN

- Una manera de aplicar Lean es utilizar Kanban.
- Es el referente principal de Lean en software.
- Sirve para gestionar cosas para las cuales Scrum no es muy útil. Es un enfoque que se usa para la gestión del cambio, no es un proceso de desarrollo ni una metodología de administración de proyecto, es un proceso para gestionar el cambio en procesos de desarrollo de software, en proyectos, pero es un ENFOQUE para la gestión de cambios.
- Kan-ban es una palabra japonesa que significa “Signal – card” o tarjeta de señal.
- A fines de 1940, Toyota comenzó a estudiar técnicas de almacenamiento y tiempo de stockeo de los supermercados y surge el concepto de **JUST IN TIME**: no tener stock, tener sólo los componentes necesarios para fabricar un producto, no más; a partir de la demanda del cliente final, puedo establecer como armar mi cadena de procesos para controlar la tasa de producción.
- KANBAN aplica la administración de colas

Principios generales KANBAN

- **Visualizar el trabajo:** Hacer el trabajo visible constantemente. Kanban se basa en el uso de tableros visuales para representar el flujo de trabajo. Estos tableros muestran las tareas pendientes, en progreso y completadas, brindando una visión clara y transparente de todo el proceso.
- **Limitar el trabajo en curso (WIP):** Kanban promueve la idea de limitar la cantidad de trabajo en curso en cualquier momento dado. Esto evita la sobrecarga del equipo y ayuda a mantener un flujo de trabajo equilibrado y constante.
- **Gestionar el flujo:** El enfoque principal de Kanban es lograr un flujo de trabajo continuo y eficiente. Se deben identificar y resolver los cuellos de botella y los problemas que afectan el flujo de trabajo, con el objetivo de optimizarlo y garantizar una entrega rápida y constante.

- **Hacer explícitas las políticas:** Kanban requiere que las políticas de trabajo y los procedimientos estén explícitos y sean claros para todos los miembros del equipo. Esto incluye definir los criterios de finalización de una tarea, las prioridades y cualquier otra regla que afecte el proceso.
- **Mejorar de forma colaborativa y evolutiva:** Kanban fomenta la mejora continua a través de la colaboración y el aprendizaje. Los equipos deben buscar constantemente formas de mejorar el proceso y adaptarlo a medida que surgen nuevas ideas y desafíos.
- **Respetar los roles, responsabilidades y habilidades existentes:** Kanban reconoce y valora las habilidades y el conocimiento existentes dentro del equipo. No impone roles predefinidos, sino que permite que los equipos se organicen según sus fortalezas individuales, siempre y cuando se respeten las responsabilidades y se logre un flujo de trabajo eficiente.

Kanban es un método cuyo propósito es mejorar procesos de desarrollo de software, permite introducir cambios en los procesos.

Nos basamos en los principios Lean

En el desarrollo de software fomenta la evolución gradual, empezando con lo que uno tiene y mejorándolo de a poco. Usando técnica de arrastre.

Como aplicar KANBAN

1. Dividir el trabajo en piezas: definir que piezas van a pasar por las colas del proceso. Las US son buenas para esto.
2. Visualizar el flujo de trabajo: Las herramientas principales son los tableros. Funciona como un pull no como un push.
Las políticas deben ser: pocas, sencillas, bien definidas, visibles, aplicables en todo momento, fácilmente modificables por los que prestan el servicio.
3. Limitar el Work In Progress: evitar que haya atascamientos. Para mantener un flujo de trabajo equilibrado.

Como aplicar KANBAN en nuestro proyecto:

- Para implementar Kanban en un proyecto primero se deben definir los objetivos y problemas que se quieren resolver. Luego se forma el equipo de trabajo y se modela el proceso en un tablero Kanban, dividido en columnas que representen las etapas del flujo de trabajo.
- Después se define la unidad de trabajo (tareas, requerimientos, user stories, etc.) y se pueden usar colores o clases de servicio para priorizar y categorizar tareas.
- También se establecen políticas de calidad y límites de trabajo en curso (WIP) para cada columna, evitando sobrecarga y acumulación de trabajo defectuoso.
- La implementación debe hacerse de manera gradual, comenzando con un tablero simple y mejorándolo con el tiempo. Es importante capacitar al equipo, fomentar la comunicación y monitorear continuamente el flujo de trabajo para detectar cuellos de botella.
- Finalmente, se deben definir actividades periódicas como planificaciones, revisiones y releases para mantener un ritmo constante y mejorar el seguimiento del proyecto.

Métricas claves de KANBAN

- **Lead Time (Vista del cliente):** como buscamos poner foco en el cliente, en el valor agregado y en la entrega rápida obtenemos esta métrica. La cual nos indica el tiempo que demora una pieza de trabajo desde que ingresa a la cola de producto hasta que está terminada. Se suele medir en días de trabajo o esfuerzo. Mide la mecánica de la capacidad del proceso. Se la llama Ritmo de Terminación.
- **Cycle Time (Vista Interna):** es el tiempo desde que el equipo empezó a trabajar con esa pieza del producto hasta que sale del ciclo. Esta métrica contiene el tiempo que se mantiene en cola. Se mide en días de trabajo. Se la llama Ritmo de Entrega.
- **Touch Time (Tiempo de Tocado):** es el tiempo el cual un ítem de trabajo fue realmente trabajado por el equipo. Se refiere a los días hábiles que paso ese ítem en columnas de WIP. Se busca que sea lo mas parecido al Cycle Time.

Lead Time: Cliente. Desde que se pide hasta que se entrega.&

Cycle Time: Equipo. Desde que se empieza a trabajar hasta que se termina el trabajo.&

Touch Time: Trabajo Real. El tiempo efectivo que se estuvo "tocando" la tarea, sin contar las esperas.

$$\text{Touch Time} \leq \text{Cycle Time} \leq \text{Lead Time}$$

Eficiencia del Ciclo de Proceso

$$\% \text{ Eficiencia ciclo proceso} = \text{Touch Time} / \text{Elapsed Time.}$$

KANBAN condensado/compactado

- Podríamos decir que las prácticas concretas son lo más fácil, es el trabajo concreto que voy a hacer para poner en práctica los principios que están planteados en el framework, la práctica es lo seguro, no me asegura que los principios se respeten, pero me ayuda a implementarlos.
- Los valores por otro lado tienen que ver con aspectos organizaciones y de cultura de la organización, sin la base de los valores es muy difícil que las practicas nos permitan implementar los principios (Tiene que ver relacionando, con esto de "hacer ágil" y "ser ágil").

Métricas de Software en los diferentes enfoques de gestión

Métricas tradicionales

- Se dividen en 3 conjuntos
 - Métricas de producto: están enfocadas en lo que se construye, son responsabilidad del equipo de desarrollo y de Testing y son particulares de ese producto. Se utilizan con propósitos técnicos y tienen como objetivo generar indicadores en tiempo real de la eficacia del análisis, el diseño, la estructura del código, la efectividad de los casos de prueba y calidad del software a construir.
 - Métricas de proyecto: Tienen su aplicación en la ejecución del proyecto que nos permite construir un determinado producto de software. En este tipo de métricas, la utilidad de la métrica termina cuando el proyecto termina. En el contexto del proyecto, la utilidad va a tener que ver con lograr corregir algo que en el proyecto no esté funcionando bien. Estas son de interés para el Líder de Proyecto y para el equipo, ya que permite ver justamente cual es el estado actual del proyecto, su salud, y que tenemos que hacer si algo no está funcionando bien. Están enfocadas a los recursos que se dedican al proyecto, como costos, esfuerzos, estimaciones y tiempo.

Las métricas de proyecto pueden dividirse en cuatro métricas básicas:

- Tamaño del producto: Métrica del producto, medir en términos concretos que tan grande o que tan pequeño es el producto que estoy construyendo.
- Esfuerzo (Horas lineales): Métrica del proceso y proyecto.
- Tiempo (Calendario): Métrica del proyecto y proceso.
- Defectos: Métrica del producto

- **Métricas de proceso (organizacionales):** Las métricas de proceso son una despersonalización de las métricas de proyecto, permiten ver en términos organizacionales como trabajamos en el conjunto de nuestros proyectos. Si para nosotros una métrica de proyecto es ver el desvío del calendario en función de lo planificado, en términos de proceso lo que hacemos es tomar muchas métricas de proyecto despersonalizando la métrica del proyecto en particular y apuntando a la organización completa, es decir, midiendo en promedio cual fue el desvío de los proyectos de la organización. Busco ver si los proyectos de la organización terminan en tiempo y forma. Si tengo un desvío general de los proyectos, todo lo que tenga que ver con las actividades de planificaciones van a tener modificaciones para ajustar ese desvío y hacer que los proyectos terminen en tiempo y forma.

Métricas en ambientes ágiles: enfocado a medir producto, resistencia a que las personas consuman tiempo para tomar métricas. Hay 2 principios ágiles que guían la elección de las métricas “la mayor prioridad es satisfacer al cliente por medio de entregas tempranas y continuas del SW valioso y funcional. “El SW funcionando es la principal medida de progreso.

- **Velocidad:** mide la cantidad de producto el PO acepto al final de una iteración. No se estima, se calcula al final del sprint review. Se utiliza para predecir la capacidad del equipo y estimar la duración del producto. Se mide en Stories Point.
- **Capacidad:** Es una estimación de cuantos Stories Points va a completar el equipo en una iteración. Cuando tengo una velocidad estable y los recursos disponibles es fácil determinar la capacidad. En los primeros sprints los puedo medir en horas lineales.
- **Running Tested Features:** Mide la cantidad de features testeadas que están funcionando, es decir, cuantas piezas de producto (historias de usuarios, casos de usos, requerimientos) se terminaron y están en ejecución. El problema de esta métrica es que no tiene en cuenta la complejidad de las piezas de producto que se implementan. Por ejemplo, si se toma como piezas de producto a historias de usuario, se mide cuántas historias de usuarios están funcionando, pero no se sabe de cuantos puntos de historias tiene cada una de ellas. Es una medición absoluta.

Métricas en KANBAN

- El foco de medición de Kanban es el proceso, ya que el sistema de trabajo de Kanban es introducir mejoras en un flujo de trabajo continuo, es decir, no existe un proyecto con principio y fin.
- Se mide el comportamiento del proceso en función al tiempo que se demoran las características.
- Las métricas son
 - **Lead time:** Métrica de vista o perspectiva del cliente. Es la más importante para el cliente. Mide desde el momento en que el cliente me pide algo (entra al Backlog) hasta que yo se lo entrego.
 - **Cycle time:** Mide el tiempo desde que el equipo comenzó a trabajar sobre una funcionalidad pedida, hasta que se lo entrega, eliminando el tiempo de espera en el Backlog. Por esto se la considera como una vista interna, que sirve al equipo de trabajo y no es tan relevante para el cliente. Es igual al Lead Time menos el tiempo que estuvo en el Backlog.
 - **Touch Time:** Mide los tiempos en los cuales las personas están efectivamente trabajando sobre una unidad de trabajo (columnas de producción), sin tener en cuenta aquellos tiempos de espera (columnas de acumulación).

$$\text{Touch Time} \leq \text{Cycle Time} \leq \text{Lead Time}$$

Tradicional

- ✓ Esfuerzo
- ✓ Tiempo
- ✓ Costos
- ✓ Riesgos

Ágil

- ✓ Velocidad
- ✓ Capacidad
- ✓ Running Tested Features

Lean

- ✓ Lead time – Elapsed time
- ✓ Cycle time
- ✓ Touch time
- ✓ Eficiencia de proceso

Métricas de Producto de Software



Defectos

- Cobertura: tratar de que sea la mayor posible para cubrir el 100% del análisis de los defectos del producto.
- Defectos por severidad: cantidad de defectos por escala de severidad de cada defecto. 5 defectos graves.
- Densidad de defectos: cuantos defectos se encuentran por historia de usuario.

UNIDAD 4

Aseguramiento de calidad de proceso y producto

Calidad

- Son todos los aspectos y características de un producto o servicio que permiten que este cumpla con todas las necesidades, tanto las que se expresan de manera clara y directa como las que están implícitas. La calidad implica la capacidad de un producto o servicio para satisfacer y superar las expectativas y requisitos del cliente.
- Estos aspectos están relacionados con la capacidad del producto para cumplir con su propósito, funcionar de manera confiable, brindar eficiencia y durabilidad, ofrecer beneficios y características adicionales que generen valor para el cliente.
- Cuando hablamos específicamente de software, las siguientes cosas hacen que el producto no tenga calidad
 - Atraso en las entregas
 - Costos excedidos
 - Falta cumplimiento de los compromisos
 - No están claros los requerimientos
 - El software no hace lo que tiene que hacer
 - Trabajo fuera de hora
 - Fenómeno del 90-90, 90% hecho 90% faltante, el líder de proyecto le pregunta a su equipo como van y cuanta falta, y su equipo le dice que bien y que falta poco.
 - ¿Dónde está ese componente?
- El aseguramiento de calidad de procesos implica establecer y mantener estándares y procedimientos claros para llevar a cabo las actividades de manera consistente y eficiente. Esto incluye definir las mejores prácticas, documentar los procesos, capacitar al personal en la ejecución correcta de las tareas y establecer controles para monitorear y medir el desempeño.
- El aseguramiento de calidad de productos se centra en garantizar que los productos cumplan con los estándares de calidad establecidos. Esto implica realizar inspecciones, pruebas y evaluaciones durante el proceso de fabricación para identificar y corregir cualquier defecto o problema antes de que el producto final sea entregado al cliente. También se pueden implementar sistemas de gestión de calidad, como ISO 9001, para asegurar que se cumplan los requisitos y se sigan las mejores prácticas.

Un software de calidad ofrece:

- Cumplimiento de las expectativas del cliente
- Cumplimiento de las expectativas del usuario
- Cumplimiento de las necesidades de la gerencia
- Cumplimiento de las necesidades del equipo de desarrollo y de mantenimiento

Principios del aseguramiento de calidad

1. La calidad no se inyecta ni se compra: La calidad no puede ser agregada o comprada después de la construcción del producto o servicio, debe estar incorporada/embebida en el producto o proceso desde el momento 0 que comienza su construcción.
2. La calidad conlleva un esfuerzo de todos: Asegurar la calidad no es solo responsabilidad de un departamento o individuo, requiere el compromiso y esfuerzo de todos los miembros del equipo.
3. Las personas son claves para lograr la calidad: El éxito en el desarrollo de software depende en gran medida de las habilidades y conocimientos del personal involucrado.
4. Se necesita un sponsor a nivel gerencial: Es importante contar con el respaldo y liderazgo de un patrocinador o sponsor a nivel gerencial que considere el desarrollo de un producto de calidad como algo fundamental y lo respalde con recursos y apoyo.
5. Se debe liderar con el ejemplo: Los líderes deben ser ejemplos de compromiso y calidad, demostrando prácticas y comportamientos que fomenten la cultura de aseguramiento de calidad en toda la organización.
6. La calidad se mide mediante el testing: Las pruebas son una herramienta fundamental para medir y controlar la calidad del software. Sin pruebas adecuadas, no se puede tener un control efectivo sobre la calidad.
7. Simplicidad, empezar por lo básico: En lugar de complicar los procesos, se debe buscar la simplicidad y comenzar por los fundamentos básicos del aseguramiento de calidad.
8. El aseguramiento de calidad debe planificarse: Es necesario contar con un plan de aseguramiento de calidad que establezca los objetivos, las actividades y los recursos necesarios para garantizar la calidad del producto o servicio.
9. El aumento de las pruebas no aumenta la calidad automáticamente: No es suficiente con aumentar las pruebas, sino que se deben aplicar técnicas y enfoques adecuados para obtener resultados eficientes.
10. Debe ser razonable para mi negocio: El aseguramiento de calidad debe adaptarse a las necesidades y características específicas de cada negocio, siendo realista y viable en términos de recursos y objetivos alcanzables.

¿Calidad para quién?

- Cuando hablamos de calidad, ya sea del proceso para construir el producto o la del producto en si mismo, es decir, la que se espera que el producto tenga, debemos saber desde que perspectiva.
- Se deben integrar todas las perspectivas, desde las expectativas de calidad que pretende el usuario, hasta las del equipo de desarrollo. Por ejemplo, el usuario quizás quiere una buena interfaz, fácil de usar, rápida. Mientras que el equipo de desarrollo va a tener calidad si es fácil de mantener y si tiene nivel de cohesión y acoplamiento adecuado.
- La visión trascendental es la que hace referencia a la pregunta de ¿para que queremos construir software? Tiene que ver con lograr cosas más allá de lo que uno se imagina que puede hacer con el software.
- El desafío en el aseguramiento de calidad es el de lograr que la calidad programada, necesaria y la lograda se alineen en la medida de lo posible (es decir, que coincidan lo mas que se pueda)

- **Calidad programada:** se refiere a las expectativas y estándares de calidad que se establecen para el producto durante su planificación y diseño. Es la calidad que se busca alcanzar y que se establece como objetivo a lo largo del proceso de desarrollo.
- **Calidad necesaria:** Es el nivel mínimo de calidad requerido para que el producto sea considerado correcto y pueda satisfacer los requerimientos y expectativas del usuario. Corresponde al conjunto de características y funcionalidades esenciales que el producto debe cumplir para cumplir con su propósito.
- **Calidad lograda:** Se refiere a la calidad real que se ha alcanzado en el producto final una vez completado el proceso de desarrollo y las pruebas correspondientes. Es el resultado concreto y medible de las actividades de desarrollo, aseguramiento y control de calidad.

Calidad en el Software

- Para desarrollar software de manera efectiva, es fundamental contar con una estructura o guía que oriente al equipo en la construcción del producto deseado. Esta estructura se conoce como **proceso**.
- Al crear o adaptar un proceso en una organización, ya sea basado definido o empírico, es recomendable basarse en Modelos de Referencia (Modelos para crearlos o Modelos de calidad). Estos modelos proporcionan lineamientos y mejores prácticas para la creación y gestión de procesos, así como para garantizar la calidad del software.
- Una vez que se ha establecido un proceso en la organización, incorporando los modelos de referencia pertinentes, es importante buscar una mejora continua del mismo. Para lograr esto, se pueden adoptar **Modelos de Mejora de Procesos**, los cuales brindan un marco de trabajo para identificar áreas de mejora y establecer proyectos que impulsen dichas mejoras.
- Además, es posible formalizar y certificar el cumplimiento de un proceso mediante **Modelos de Evaluación**. Estos modelos se utilizan para medir y evaluar el grado de adherencia del proceso a los estándares y modelos de referencia establecidos.
- El proceso se materializa en proyectos, los cuales son la unidad de trabajo que da vida a dicho proceso.
- El proyecto a su vez es el ámbito donde uno trabaja para obtener el producto, la versión de producto que yo someto a evaluación se va generando en el contexto de un proyecto. Existen técnicas y herramientas tanto para el aseguramiento de calidad, como la revisión de pares o las auditorías (De configuración, auditorías físicas y funcionales), como para el control de calidad, como es el Testing.
- Cuando yo hago actividades de aseguramiento de calidad tanto de proceso como de producto, se hace en el contexto de un proyecto en específico.
- El **aseguramiento de calidad de proceso** se enfoca en el proceso de desarrollo de software y busca mejorar la forma en que se trabaja, mientras que el **aseguramiento de calidad de producto** se centra en evaluar y garantizar la calidad del producto de software resultante.

	ASEGURAMIENTO DE CALIDAD DE PROCESO	ASEGURAMIENTO DE CALIDAD DE PRODUCTO
FOCO	Se concentra en evaluar y mejorar el proceso de desarrollo de software en sí mismo, es decir, la forma en que las actividades se llevan a cabo.	Se enfoca en evaluar y garantizar la calidad del producto de software resultante, es decir, los artefactos y entregables generados durante el proceso de desarrollo.
OBJETIVO	Busca establecer prácticas efectivas, eficientes y consistentes para el desarrollo de software, con el fin de optimizar la productividad y minimizar los errores o problemas en el proceso	Busca asegurar que el producto de software cumpla con los requisitos establecidos, sea confiable, funcione correctamente y sea adecuado para su uso previsto.
ACT. CLAVE	Involucra definir estándares y buenas prácticas, realizar revisiones técnicas y auditorías de proceso, medir y monitorear indicadores de calidad del proceso, y gestionar la configuración del software.	Incluye realizar pruebas de software (como pruebas funcionales, de rendimiento o de seguridad), realizar revisiones de código, realizar auditorías de producto, verificar el cumplimiento de estándares de calidad y realizar validaciones con los usuarios o stakeholders del producto.

Calidad en procesos definidos vs Calidad en procesos empíricos

- El problema de visión entre los procesos definidos y empíricos se debe a diferentes enfoques sobre la relación entre la calidad del proceso y la calidad del producto.
- En el caso de los procesos definidos, se argumenta que, si el proceso en sí mismo es de alta calidad y se respeta en el contexto del proyecto, entonces el producto resultante también será de alta calidad. Ya que un proceso sólido y bien estructurado conduce generalmente a resultados buenos.
- En el caso de procesos empíricos la calidad del producto depende de las personas que participan en el desarrollo. Es decir, si el equipo hace bien las tareas y cumple con las responsabilidades, el producto final tendrá calidad.

Calidad en el Producto

- La calidad del producto No se puede sistematizar, no hay modelos en la industria generalizados para evaluar calidad de producto que se puedan aplicar como una plantilla a todos los productos igualmente.
- Su aseguramiento se hace mediante las **Revisiones Técnicas** (Revisiones de pares) y **Auditorias**, mientras que su control se hace mediante el **Testing**.

Modelos de calidad de producto

- Son marcos o estructuras conceptuales que se utilizan para evaluar y medir la calidad de los productos de software. Proporcionan un conjunto de características, criterios y métricas que se utilizan para determinar la calidad de un producto de software.
 - Modelo de Barbacci: Este modelo ofrece una perspectiva amplia y completa de la calidad del producto de software, centrándose en varios aspectos clave. La calidad del producto se mide mediante su confiabilidad, performance y seguridad. Debemos trabajar para lograr un equilibrio entre estas tres cosas.
 - Modelo de McCall: La calidad depende de tres factores determinantes que causalmente tiene que ver con las visiones de calidad mencionadas anteriormente:
 - Revisión del producto: Es la capacidad de verificación del producto. Facilidad de mantenimiento, flexibilidad y prueba.
 - Operación del producto: Es la calidad del producto en uso, lo que mas le importa al usuario final. Corrección, fiabilidad y usabilidad.
 - Transición del producto: Facilidad con la que el producto puede expandirse y crecer.
- Cuando hablamos de calidad de producto si queremos hacer aseguramiento es con revisiones técnica o auditorias y para controlar la calidad es con testing (visibiliza que tan cerca o lejos estamos de los requerimientos que se plantearon para el producto).

Calidad en el proceso

- El proceso es el único factor controlable para mejorar la calidad del Software y el rendimiento como organización.
- El producto, las personas, la tecnología, las características del cliente, las condiciones de negocio y el entorno de desarrollo son incontrolables.
- A raíz de un proceso que adopte la calidad, se puede instanciar un proyecto que tenga la calidad como factor embebido, lo que lleva a un Software de calidad.
- El Aseguramiento de calidad de Software implica insertar, en cada una de las actividades, acciones que detecten lo más temprano posible oportunidades de mejora, tanto sobre el producto como sobre el proceso.
- Implica la definición de estándares y procesos de calidad apropiados, y el aseguramiento de que los mismos sean respetados.

- Debe ayudar a desarrollar una cultura de calidad, donde la calidad es vista como una responsabilidad de todos y cada uno.

Definición de un Proceso de Software

- **Proceso:** Secuencia de pasos ejecutados para un propósito dado (IEEE)
- **Proceso de Software:** Un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados (Sw-CMM)
- El proceso no es solamente las tareas escritas, sino que se define como una mesa de 3 patas, donde sí es cierto que debe haber lineamientos de cómo vamos a trabajar (**procedimientos**), pero debe haber **personas con habilidades con entrenamiento y motivación**, y el mejor soporte de **herramientas automatizadas y equipamiento**.

¿Cómo es un proceso para Construir Software?



- La ingeniería se le llama a la etapa de requerimientos, análisis y diseño. Implementación, es la codificación, el testing y el despliegue. Ahora incorporamos en un proceso de desarrollo de software disciplinas de gestión y de soporte estas son:
 - Planificación y Seguimiento de Proyectos
 - Administración de Configuraciones
 - Aseguramiento de la Calidad

Aseguramiento de calidad de Software

- Es referido a asegurar que se alcancen los niveles requeridos de calidad para el producto de software. Implica la definición de estándares y procesos de calidad apropiados y asegurar que los mismos sean respetados.
- Debería ayudar a desarrollar una “cultura de calidad” donde la calidad es vista como una responsabilidad de todos y cada uno.
- Es insertar en cada una de las actividades acciones tendientes a detectar lo más tempranamente posible sobre el producto y sobre el proceso

Reporte del grupo de Aseguramiento de Calidad (GAC)

- Para materializarlo en forma concreta, cuando una organización quiere un grupo de aseguramiento de la calidad hay que tener en cuenta las siguientes consideraciones:
 - No se debe reportar al líder de proyecto, como forma de mantener la independencia y la objetividad.
 - El grupo de aseguramiento de calidad tiene que tener un reporte independiente al reporte de los proyectos y tiene que ser lo más cerca posible a depender directamente del nivel más alto de la organización.

- No debe existir más de un nivel de gerencia entre la Dirección y el GAC, el reporte debe ser directo a la gerencia de dirección.
- El GAC debe reportar a alguien realmente interesado en la calidad del Software.

Actividades de la Administración de Calidad de Software

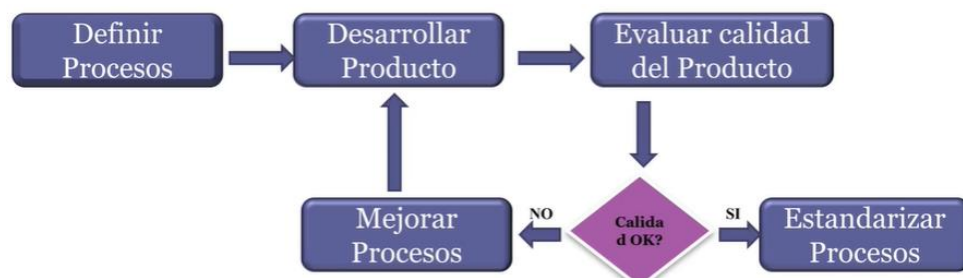
- Aseguramiento de calidad es insertarse en el proceso de desarrollo de software mientras el software se está construyendo.
- **ASEGURAMIENTO DE CALIDAD:** Establecer estándares y procedimientos de calidad. Elegir contra que estándares se van a efectuar las comparaciones. (Las revisiones técnicas y las auditorías son actividades de comparación)
- **PLANIFICACIÓN DE CALIDAD:** Se seleccionan los procedimientos y estándares de calidad para un proyecto y particular, y se los modifica si fuera necesario. Ejemplo, en qué momento se van a hacer las revisiones, las auditorías.
- **CONTROL DE CALIDAD:** Se ejecuta el control de calidad de acuerdo con los procedimientos y estándares de calidad elegidos. Se ejecutan las actividades definidas en la planificación para ver en qué situación está el proyecto.

Funciones del Aseguramiento de Calidad de Software

- Prácticas de Aseguramiento de Calidad
Desarrollo de herramientas adecuadas, técnicas, métodos y estándares que estén disponible para realizar las revisiones de Aseguramiento de Calidad.
- Evaluación de la planificación del Proyecto de Software
- Evaluación de Requerimientos
- Evaluación del Proceso de Diseño
- Evaluación de las prácticas de programación
- Evaluación del proceso de integración y prueba de software
- Evaluación de los procesos de planificación y control de proyectos
- Adaptación de los procedimientos de Aseguramiento de calidad para cada proyecto.

Procesos basados en calidad

- El trabajo de las personas encargadas de la calidad en el desarrollo de software implica definir y establecer los procesos que se seguirán para garantizar la calidad del producto final. Estos procesos incluyen actividades como la planificación, el diseño, la implementación, las pruebas y la entrega del software.
- Una vez que se han establecido los procesos, es importante evaluar continuamente su efectividad y calidad. Esto se logra a través de la evaluación del producto de software resultante. Se analizan los artefactos y entregables generados para determinar si cumplen con los estándares de calidad y si satisfacen los requisitos establecidos.
- En función de los resultados de la evaluación del producto, se toman decisiones sobre la mejora del proceso. Si se identifican deficiencias o problemas en el producto, se realiza una revisión exhaustiva del proceso correspondiente y se implementan mejoras para corregir los errores y evitar que se repitan en el futuro.



- Por otro lado, si el producto de software cumple con los estándares de calidad y se considera exitoso, el proceso utilizado se estandariza. Esto implica documentar y comunicar de manera efectiva los pasos, las actividades y las buenas prácticas empleadas durante el desarrollo del producto. Estos procesos estandarizados se distribuyen al resto de la organización para que puedan ser seguidos y aplicados de manera consistente en proyectos futuros.

Modelos de Mejora

- Los modelos de mejora son recomendaciones o estándares para encarar un proyecto de mejora de un proceso.
- El propósito de un modelo de mejora es tomar el proceso de una organización y elaborar un proyecto, cuyo resultado va a ser un proceso mejorado.
- Sirven para armar un proyecto que nos va a permitir a nosotros tener un nuevo proceso mejorado para aplicar en los nuevos proyectos de desarrollo de la organización.
- Algunos modelos para la mejora de procesos son:
 - SPICE: Software Process Improvement Capability Evaluation
 - IDEAL: Initiating, Diagnosing, Establishing, Acting, Leveraging

IDEAL

- Modelo de mejora que nos sirve para definir en una organización, un proyecto que ayude a mejorar el proceso que esa empresa tiene.
- Es un proceso cíclico ya que está orientado a la mejora continua
- INITIATING (Inicio): Se busca un sponsoreo en la organización. Esto es importante, puesto que los proyectos de mejoras de proceso no suelen ser tomados como prioridad a menudo en las organizaciones.
- DIAGNOSING (Diagnostico): Se realiza un diagnóstico para determinar cuál es el punto de partida del proyecto. Se realiza un Análisis de brecha que sirve para ver donde está parada la organización y a qué objetivo quiere apuntar.
- ESTABLISHING (Establecimiento): Se planifica y se establece un plan de mejora detallado. Se definen los objetivos, se identifican las actividades necesarias y se asignan los recursos necesarios.
- ACTING (Actuar): Se ejecuta el plan de acción y las estrategias definidas. Probamos el proceso definido en algún proyecto (pruebas piloto).
- LEARNING (Aprender): Si el resultado es positivo, lo extrapolamos a toda la organización. Se documentan los resultados del plan de acción y se vuelve a ejecutar el modelo con las lecciones aprendidas y para identificar oportunidades de mejora.

Modelos de Calidad

- Se usan como referencia para bajar lineamientos que el proceso debe seguir para llegar a un cierto objetivo.

Modelo de calidad CMMI (Capability Maturity Model Integration)

- Surgió específicamente para empresas, organizaciones que hacen específicamente software. Se usa de referencia en base a los objetivos que quieren alcanzarse. El CMMI dice el **qué** no el **cómo**, es un modelo descriptivo no prescriptivo.
- El objetivo principal de CMMI es proporcionar una guía para mejorar la capacidad y madurez de los procesos organizacionales. Se basa en buenas prácticas recopiladas de la industria y está diseñado para ayudar a las organizaciones a alcanzar niveles superiores de calidad, eficiencia y satisfacción del cliente.

- CMMI se estructura en niveles de madurez y áreas de proceso.
 - Los niveles de madurez representan etapas de mejora evolutiva en la organización, desde un nivel inicial ad hoc hasta un nivel de optimización continuo.
 - Las áreas de proceso se enfocan en aspectos específicos del desarrollo y gestión de software, como la planificación, el monitoreo y control, la gestión de requisitos, el diseño, la implementación y la evaluación.
- CMMI proporciona un conjunto de prácticas y criterios que las organizaciones pueden seguir para mejorar sus procesos. Son evaluados mediante una evaluación formal realizada por profesionales capacitados, que permiten obtener una medida objetiva de la madurez y capacidad de los procesos organizacionales.
- Al seguir este modelo, las organizaciones pueden lograr beneficios como la reducción de defectos, la mejora de la productividad, la gestión de riesgos más efectiva, la alineación con los objetivos del negocio y la mejora en la entrega de productos y servicios.
- Es un modelo flexible y adaptable, que se puede personalizar según las necesidades y características de cada organización.
- Tiene 3 ámbitos de mejora
 - **CMMI DEV:** Provee la guía para medir, monitorizar y administrar los procesos de desarrollo. Específicamente en el desarrollo de Software
 - **CMMI SVC:** Provee la guía para entregar servicios, internos o externos.
 - **CMMI ACQ:** Provee la guía para administrar, seleccionar y adquirir productos o servicios.

Representaciones CMMI

Hay 2 formas de mejorar, evolucionar con el proceso

1. Por etapas:

- En esta representación, se organizan los niveles de madurez de la organización en etapas predefinidas. Cada nivel de madurez representa un conjunto de prácticas y capacidades específicas que deben alcanzarse para avanzar al siguiente nivel.
- Los niveles de madurez son:
 - ❖ Nivel 1: Inicial: La organización tiene procesos ad hoc y no estandarizados.
 - ❖ Nivel 2: Gestionado: La organización establece prácticas de gestión básicas y utiliza enfoques más disciplinados para el desarrollo de proyectos.
 - ❖ Nivel 3: Definido: La organización establece y documenta procesos estandarizados y repetibles.
 - ❖ Nivel 4: Cuantitativamente gestionado: La organización establece mediciones cuantitativas para el control y mejora de los procesos.
 - ❖ Nivel 5: Optimizado: La organización enfoca en la mejora continua y la optimización de los procesos.
- Divide a las organizaciones en dos tipos: maduras o inmaduras
- Las organizaciones inmaduras son las organizaciones de nivel 1.
- Las organizaciones maduras son las organizaciones de nivel 2 a 5.
- Mientras más madura es la organización, más capacidad tiene para cumplir con los objetivos, entonces mejoraba la calidad de sus productos y disminuía sus riesgos.
- La ventaja de esta representación es que provee una única clasificación que facilita las comparaciones entre organizaciones. Como así también proporciona una estructura clara y lineal para mejorar la madurez de la organización.

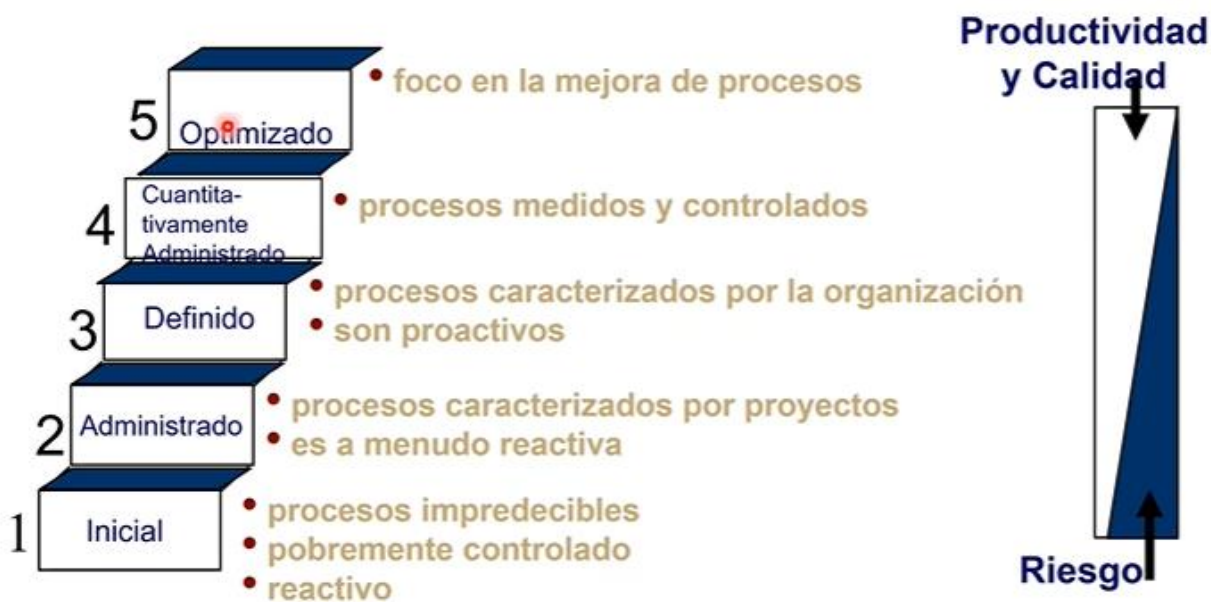
2. Continua

- Esta representación lo que hace es elegir áreas de proceso dentro de las que el modelo te ofrece y yo elijo cuales son los procesos quiero mejorar por separado, entonces en lugar de medir la madurez de toda la organización, se mide la capacidad de un proceso en particular.
- Esta representación mide la CAPACIDAD de las distintas áreas de proceso para poder cumplir los objetivos.
- Mide áreas individuales
- Cada área de proceso tiene metas y prácticas específicas asociadas. La evaluación de la capacidad se realiza en una escala de 0 a 5, donde cada nivel de capacidad indica el grado de implementación y desempeño de las prácticas específicas de esa área.
- Permite a las organizaciones seleccionar y mejorar áreas de proceso específicas de acuerdo con sus necesidades y prioridades. Se puede trabajar en paralelo en diferentes áreas de proceso sin tener que seguir una secuencia fija.

CMMI Roles y Grupos

- Cuando CMMI se refiere a grupos se refiere a la existencia de roles que cubren ciertas tareas. Esos roles se adaptan al tamaño de la organización y a las expectativas de madurez que la organización quiere llegar.
- Lo importante es que exista alguien responsable de cubrir las actividades de cada uno de los roles o grupos.

CMMI Representación por etapas



- Nivel 1: No existe visibilidad respecto del proceso. No se sabe cuándo termina, cuánto cuesta ni la calidad de lo que se obtiene. Son organizaciones reactivas (atacan el problema una vez que sucedió en lugar de prevenirlo) e inmaduras.
- Nivel 2: la organización comienza a tener controles básicos para la gestión de proyectos y procesos. Se busca una mayor estandarización y se inicia la recopilación de métricas para el seguimiento y la toma de decisiones.
- Nivel 3: la organización ha establecido y documentado procesos definidos y estandarizados. Se definen roles y responsabilidades, y existe retroalimentación.
- Nivel 4: la organización tiene la capacidad de cuantificar y medir su rendimiento. Se establecen métricas, y se recopilan datos para analizar el desempeño de los procesos y procedimientos.
- Nivel 5: la organización se enfoca en la mejora continua y en la optimización de sus procesos.

En la materia se hace enfoque en el nivel 2. Si alcanzo un nivel 2 de CMMI puedo decir que mi organización tiene madurez para administrar sus proyectos y que el resultado de sus proyectos va a ser un producto de software de lo que se sabe que se espera de él.

RESUMEN DE LO ANTERIOR

- Si se quiere mejorar el proceso de una organización y se elige IDEAL como modelo de mejora y marco de referencia para implementar la mejora y queremos alcanzar un modelo de calidad CMMI. Cuando el proceso esté listo, definido y que haya proyectos que lo hayan usado, entonces aparecen los Modelos para evaluar, en el cual un grupo de personas evalúan lo realizado (son instancias de auditoria o certificaciones), el método formal para evaluar y que una empresa sepa si adquirió un determinado nivel o capacidad de CMMI se llama SCAMPI.

Intereses fundamentales de la gestión de calidad

- A nivel de **organización**, la gestión de calidad se ocupa de establecer un marco de proceso y estándares de organización que conducirán a software de mejor calidad. Esto supone que el equipo de gestión de calidad debe tener la responsabilidad de definir los procesos de desarrollo del software a usar, los estándares que deben aplicarse al software y la documentación relacionada, incluyendo los requerimientos, el diseño y el código del sistema.
- A nivel del **proceso**, la gestión de calidad implica la aplicación de procesos específicos de calidad y la verificación de que continúen dichos procesos planeados, garantizando que los resultados del proyecto estén en conformidad con los estándares aplicables a dicho proyecto.
- A nivel del **proyecto**, la gestión de calidad se ocupa también de establecer un plan de calidad para un proyecto. El plan de calidad debe establecer metas de calidad para el proyecto y definir cuáles procesos y estándares se usarán.

Puntos clave

- El software puede analizarse desde varias perspectivas: como proceso, como producto, la calidad también.
- La calidad del software es difícil de medir.
- El software como proceso es el fundamento para mejorar la calidad.
- Trabajar con calidad es más barato.
- La mejora de procesos exitosa requiere compromiso y cambio organizacionales.
- Existen varios modelos disponibles para dar soporte a los esfuerzos de mejora.
- La mejora de procesos en el software ha demostrado retornos de inversión sustanciales.

CMMI cara a cara con Ágil

1. Filosofía y enfoque:

- **CMMI**: se basa en un enfoque más tradicional y orientado a procesos. Se enfoca en establecer prácticas estandarizadas, mejorar la madurez de los procesos y lograr la calidad a través de la planificación y el control rigurosos.
- **Ágil**: se basa en principios y valores ágiles, como la adaptabilidad, la colaboración y la entrega de valor incremental. Se enfoca en la flexibilidad, la respuesta al cambio y la entrega temprana de software funcional.

2. Gestión de proyectos:

- **CMMI:** se centra en la gestión de proyectos a través de procesos planificados y controlados. Se pone énfasis en la definición y el seguimiento de requisitos, la planificación detallada, el monitoreo del progreso y el control de cambios.
- **Ágil:** promueve la gestión de proyectos mediante iteraciones cortas y frecuentes. Se enfoca en la colaboración con el cliente, la retroalimentación continua y la adaptabilidad a medida que se desarrolla el producto. La planificación es flexible y se ajusta a medida que se obtiene más información y se responden a los cambios.

3. Entrega de software:

- **CMMI:** tiende a enfocarse en la entrega final del software, centrándose en los aspectos de calidad, cumplimiento de requisitos y procesos establecidos.
- **Ágil:** se centra en la entrega temprana y continua de software funcional y de alta calidad. Se valora la retroalimentación del cliente y la adaptación del producto a medida que se desarrolla, lo que permite una mayor satisfacción del cliente y la capacidad de responder rápidamente a los cambios en los requisitos.

4. Flexibilidad y adaptabilidad:

- **CMMI:** proporciona un marco estructurado y detallado de prácticas y procesos que deben seguirse. Si bien se puede personalizar, tiene un enfoque más rígido y menos adaptable a los cambios.
- **Ágil:** se basa en la adaptabilidad y la flexibilidad para responder a los cambios en los requisitos y las necesidades del cliente. Los equipos ágiles tienen la capacidad de ajustar su enfoque y prioridades según sea necesario.

Diferencias

“Valores
esenciales”



CMMI	MÉTODOS ÁGILES
→ Medir y mejorar el proceso [Mejores Procesos ↓ Mejor Producto]	→ Respuestas a clientes → Mínima sobrecarga → Refinamiento de Requerimientos - Metáforas - Casos de negocio
→ Características de las personas - Disciplinados - Siguen reglas - Aversión al riesgo	→ Características de las personas - Comfortable - Creative - Risk Takers
→ Comunicación - Organizacional - Macro	→ Comunicación - Person to Person - Micro
→ Gestión de Conocimiento - Activos de proceso	→ Gestión de Conocimiento - Personas

AUDITORIAS

Auditorias de Software

Son evaluaciones independientes de los productos o procesos de Software para asegurar el cumplimiento con estándares, lineamientos, especificaciones y procedimientos, basada en un criterio objetivo incluyendo documentación que especifique:

- La forma o contenido de los productos a ser desarrollados
- El proceso por el cual los productos van a ser desarrollados.
- Como debería medirse el cumplimiento con estándares o lineamientos.

¿Por qué auditar?

- Porque se da una opinión objetiva e independiente
- Porque permite identificar áreas de insatisfacción potencial del cliente
- Porque nos permite asegurar al cliente que estamos cumpliendo con nuestras expectativas
- Porque permite identificar oportunidades de mejora.

Beneficios de las auditorías de calidad de software

- Evaluar el cumplimiento del proceso de desarrollo
- Determinar la implementación efectiva de:
- El proceso de desarrollo organizacional
- El proceso de desarrollo del proyecto
- Las actividades de soporte
- Dar visibilidad a la gerencia sobre los procesos de trabajo

Resultado: Mejores productos conllevan a clientes satisfechos y crecimiento del negocio

Tipos de auditorías de software

1. AUDITORIA DE PROYECTO

Valida el cumplimiento del proceso de desarrollo. Es responsable de evaluar si el proyecto se ejecutó en base al proceso que se dijo que se iba a ejecutar. Apunta a ver el nivel de cumplimiento del proceso que como equipo nos comprometimos a utilizar.

Se llevan a cabo de acuerdo a lo establecido en el **PACS** (Planeamiento de Aseguramiento de Calidad de Software). El PACS debería indicar la **persona responsable de realizar esta auditoría**.

Las inspecciones de Software y las revisiones de documentación de diseño y prueba deberían incluirse en esta auditoría.

AUDITADO (suele ser el líder de proyecto)	AUDITOR (debe ser de fuera del proyecto)
• Acuerda la fecha de auditoría • Participa en la auditoría • Proporciona evidencia al auditor • Contesta al reporte de auditoría • Propone el plan de acción para deficiencias citadas en el reporte • Comunica el cumplimiento del plan de acción.	• Acuerda la fecha de la auditoría. • Comunica el alcance de la auditoría, • Recolecta y analiza la evidencia objetiva que es relevante y suficiente para tomar conclusiones acerca del proyecto auditado, • Realiza la auditoría • Prepara el reporte • Realiza el seguimiento de los planes de acción acordados con el auditado

GERENTE DE SQA (responsable de manejar a las personas que tiene a su cargo que hacen auditorías)

- Prepara el plan de auditoría
- Calcula el costo de las auditorías
- Asigna los recursos
- Responsable de resolver las no conformidades.

El objetivo de esta auditoría es verificar objetivamente la **consistencia del producto** a medida que evoluciona a lo largo del proceso de desarrollo, determinando que:

Las interfaces de hardware y software sean **consistentes con los requerimientos de diseño en la ERS**.

Los **requerimientos funcionales de la ERS** se validan de en Plan de Verificación y Validación de Software.

El **diseño del producto**, a medida que DDS (Sistema de diagnóstico digital) evoluciona, satisface los requerimientos funcionales de la ERS.

El código es consistente con el DDS.

2. AUDITORIA DE CONFIGURACIÓN FUNCIONAL → Valida que el producto cumpla con los requerimientos

compara el software que se ha construido (incluyendo sus formas ejecutables y su documentación disponible) con los requerimientos de software especificados en la ERS.

El propósito de la auditoría funcional es asegurar que el código implementa sólo y completamente los requerimientos y las capacidades funcionales descriptos en la ERS.

El responsable de QA deberá validar si la matriz de rastreabilidad está actualizada.

3. AUDITORIA DE CONFIGURACIÓN FÍSICA → Valida que el ítem de configuración tal como está construido cumpla con la documentación técnica que lo describe.

Su propósito es asegurar que la documentación que se entregará es consistente y describe correctamente al código desarrollado.

El PACS debería indicar la persona responsable de realizar la auditoría física.

El software podrá entregarse sólo cuando se hayan arreglado las desviaciones encontradas.

Proceso de auditoría

Preparación y planificación: Se planifica y prepara la auditoría en forma conjunta entre el auditado y el auditor. Generalmente es el líder de proyecto quien la convoca. Estas no son sorpresas.

Ejecución: Durante la ejecución, el auditor pide documentación y hace preguntas. Busca evidencia objetiva (lo que está documentado), y subjetiva (lo que el equipo expresa que hace).

Análisis y reporte de resultado: Se analiza la documentación, se prepara un reporte y se lo entrega al auditado.

Seguimiento: Dependiendo de cómo funcione el acuerdo entre el auditado y el auditor, el auditor puede hacer un seguimiento de las desviaciones que encontró hasta que considere que han sido resueltas.

CHECKLIST de auditoría con preguntas tipo para garantizar que independientemente de quien es el que hace la auditoría el foco de las cosas que se controlan sea el mismo.



- Fecha de la auditoría
- Lista de auditados (identificando el rol)
- Nombre del auditor
- Nombre del proyecto
- Fase actual del proyecto (si aplica)
- Objetivo y alcance de la auditoría
- Lista de preguntas

Lista de resultados de una auditoría

- **Buenas prácticas:** practica, procedimiento o instrucción que se ha desarrollado mucho mejor de lo esperado. Cuando el auditor encuentra algo superador a lo que se esperaba. Cuando se hace un informe de auditorías generalmente se comienzan con estas buenas prácticas.
- **Desviaciones:** requieren un plan de acción por parte del auditado. Cualquier cosa que no se hizo como el proceso indicaba que debía hacerse.
- **Observaciones:** condiciones que deberían mejorarse pero que no requieren un plan de acción. Cosas que advierte el auditor que no llegan a ser desviaciones, pero son riesgosas porque pueden llegar a traer un problema, la destaca por si el equipo quiere considerarlas para mejorarlas.

Revisiones técnicas

Verificación y validación

- Es un proceso de ciclo de vida completo.
- Inicia con las revisiones de los requerimientos y continúa con las revisiones del diseño, inspecciones del código hasta la prueba.
- Validación: ¿Estamos construyendo el producto correcto?
- Verificación: ¿Estamos construyendo el producto correctamente?

Falla: error en un producto de trabajo.

Producto de trabajo: salida de cualquier actividad correspondiente al ciclo de vida de desarrollo.

¿Por qué existen las fallas?

- Problemas de comunicación.
- Limitaciones de memoria.

Principios

- Prevenir es mejor que curar.
- Evitar es más efectivo que eliminar.
- La retroalimentación enseña efectivamente.
- Priorizar lo rentable.
- Olvidarse de la perfección, no se puede conseguir.
- Enseñar a pescar, en lugar de dar el pescado.

Existen dos aproximaciones complementarias:

- Revisiones técnicas.
- Pruebas de Software.

Revisiones técnicas – Peer Review

Una **revisión técnica** o **Peer Review** es una actividad realizada por un colega, cuyo propósito es mejorar la calidad del software mediante la detección temprana de errores en cualquier artefacto generado durante el proyecto.

Estos artefactos pueden ser, por ejemplo:

- Código.
- Requerimientos.
- Diseño.
- Arquitectura.
- Riesgos.
- Estimaciones.
- Planes.

Es un proceso **estático de validación y verificación**, ya que permite detectar errores, pero no corregirlos directamente.

Objetivos principales

Los objetivos de una revisión técnica son:

- Introducir el concepto de verificación y validación.
- Evitar el retrabajo.
- Motivar a realizar un mejor trabajo.
- Detectar errores de manera temprana.
- Mejorar la calidad del software.

En una revisión técnica **nunca se juzga al autor del artefacto**, sino únicamente al artefacto en sí. Para que la revisión sea efectiva, es necesario que los errores puedan ser revelados entre los miembros del equipo sin buscar culpables.

Por eso, se debe desarrollar una cultura de trabajo basada en el apoyo y la colaboración.

Esta actividad trascendió metodologías y filosofías específicas. Incluso en enfoques ágiles se utiliza para transformar auditorías en revisiones entre pares, evitando traer personas externas al equipo.

Por ejemplo, un programador junior puede solicitar a un programador senior una revisión técnica sobre un patrón de diseño arquitectónico que desea implementar.

Ventajas y desventajas

Ventajas

- Pueden descubrirse muchos errores.
- Pueden inspeccionarse versiones incompletas.
- Pueden considerarse otros atributos de calidad.

Desventajas

- Es difícil introducir inspecciones formales.
- Al inicio, aumentan los costos y recién generan ahorro cuando los equipos adquieren experiencia.
- Requieren tiempo para organizarse.
- Pueden parecer que ralentizan el proceso de desarrollo.

Tipos de revisiones

Las revisiones técnicas pueden ser:

Revisiones formales → Tienen un proceso definido y roles específicos.

Ejemplo:

- Inspecciones.
- Inspección de código de Fagan.
- Inspección de Gilb.

Revisiones informales → No tienen un proceso formal definido.

Ejemplo:

- Walkthrough o recorrido.

Walkthrough o recorrido → El **walkthrough** es una técnica de análisis estático informal.

En esta técnica, un diseñador o programador guía a miembros del equipo de desarrollo y otras partes interesadas a través de un producto de software.

Durante el recorrido, los participantes formulan preguntas y realizan comentarios sobre posibles errores, violaciones de estándares de desarrollo u otros problemas.

No existe un proceso formal. Consiste en reuniones informales entre colegas donde se debaten correcciones a aplicar sobre el producto de trabajo.

Objetivos del walkthrough

Sus principales objetivos son:

- Mínima sobrecarga.
- Capacitación de desarrolladores.
- Rápido retorno.

Esta técnica no obtiene métricas para aprender ni dejar registros. Sin embargo, es una de las técnicas más elegidas en enfoques ágiles.

En estos enfoques suele realizarse una revisión entre colegas después de completar cada iteración del software, donde se pueden exponer conflictos y problemas de calidad del producto.

Inspecciones → Las **inspecciones** son revisiones formales.

Tienen un proceso definido y cuentan con un conjunto de roles. Además, requieren el uso de un **checklist**, que ayuda a recordar qué aspectos se deben controlar.

En las inspecciones se toman métricas y, al finalizar, se realiza un reporte de revisión para analizar los defectos encontrados.

Es una actividad que contribuye a garantizar la calidad del software, y su éxito depende de una buena planificación.

Objetivos de las inspecciones

Las inspecciones tienen como objetivos:

1. Descubrir errores.
2. Verificar que el software alcanza sus requerimientos.
3. Garantizar que el software fue construido de acuerdo con ciertos estándares.
4. Conseguir un software desarrollado de manera uniforme.
5. Hacer que los proyectos sean más manejables.

Las inspecciones son procesos de **time boxing** y exigen un alto esfuerzo intelectual.

Roles participantes en una inspección → Al ser una técnica formal, una inspección debe contar con ciertos roles.

1. Autor

Es el creador o encargado de mantener el producto a inspeccionar.

Inicia el proceso seleccionando a un moderador y, junto con él, elige al resto de los roles. También entrega el producto a inspeccionar al moderador.

2. Moderador

Es quien planifica y lidera la revisión.

Trabaja junto al autor para elegir los demás roles. Entrega el producto a inspeccionar al inspector dos días antes de la reunión.

Además, coordina la reunión para evitar conductas inapropiadas y realiza el seguimiento de los defectos encontrados.

3. Anotador

Registra los hallazgos de la inspección.

Usualmente, también termina confeccionando el reporte de la revisión.

4. Lector

Lee el producto que será inspeccionado.

Este rol es necesario para evitar que los participantes se dispersen durante la reunión.

5. Inspector

Examina el producto antes de la reunión para encontrar defectos.

También registra sus tiempos de preparación. Todos los participantes pueden ser inspectores.

Algunos roles pueden ser asumidos por la misma persona.

Etapas del proceso de inspección

1. Planificación

El moderador, a pedido del autor, planifica la inspección.

Define:

- Lugar.
- Tiempo de duración.
- Roles participantes.

La duración de las reuniones no debe superar las dos horas, ya que la inspección implica un alto esfuerzo intelectual.

2. Visión general (Esta etapa es opcional.)

- El autor realiza una descripción general del producto que será inspeccionado.

3. Preparación

Cada rol se prepara para la reunión.

- Cada participante recibe una copia del producto de trabajo, que deberá leer y analizar con el objetivo de encontrar posibles defectos.
- Esta preparación permite que la reunión de inspección sea más productiva.

4. Reunión de inspección

El equipo realiza un análisis para recolectar los posibles defectos encontrados previamente y descartar falsos positivos.

Durante la reunión:

- El lector lee el producto de trabajo.
- Los inspectores comparten los defectos encontrados.
- El anotador registra los hallazgos.

La reunión finaliza con una conclusión sobre si se acepta o no el producto de trabajo inspeccionado.

Finalmente, se realiza un informe detallando:

- Qué se revisó.
- Quiénes participaron.
- Qué se descubrió.
- Qué se concluyó.

5. Corrección

Finalizada la reunión, el autor realiza las correcciones de los defectos encontrados.

6. Seguimiento

Dependiendo de la gravedad de los defectos, puede existir un proceso de reinspección.

Si los defectos son graves, se realiza una nueva inspección.

Si el defecto es simple, el autor simplemente lo corrige.

En otros casos, puede ser suficiente que el autor se reúna únicamente con el moderador para revisar las correcciones realizadas.

Definición de estándares

Un aspecto importante del aseguramiento de la calidad es la definición o selección de estándares que deben aplicarse al proceso de desarrollo de software o al producto de software.

Los estándares son importantes porque permiten establecer buenas prácticas, definir criterios de calidad y asegurar que todos los integrantes de una organización trabajen de manera uniforme.

Importancia de los estándares

Los estándares son importantes por tres razones principales:

1. Reutilizan conocimiento y experiencia previa

Se basan en el conocimiento sobre las mejores prácticas para la organización. Muchas veces este conocimiento se obtiene después de pruebas y errores, por lo que convertirlo en estándar ayuda a evitar errores del pasado.

2. Permiten evaluar la calidad

Al usar estándares, se establece una base para decidir si se alcanzó el nivel de calidad requerido. Esto depende de que los estándares reflejen las expectativas del usuario en aspectos como confiabilidad, usabilidad y rendimiento.

3. Unifican la forma de trabajo

Los estándares aseguran que todos los trabajadores de una organización adopten las mismas prácticas. Esto reduce el esfuerzo de aprendizaje al iniciar un nuevo proyecto o trabajo.

Tipos de estándares en calidad de software

Para la gestión de calidad de software se utilizan dos tipos de estándares:

Estándares de producto

Se aplican al producto de software que se va a desarrollar.

Incluyen, por ejemplo:

- Estándares de documentos.
- Estándares de codificación.

Estándares de proceso

Establecen los procesos que deben seguirse durante el desarrollo.

Incluyen, por ejemplo:

- Definición de requerimientos.
- Procesos de diseño.
- Procesos de validación.

TESTING

Contexto

- El testing es una tarea que se realiza dentro del ámbito del **aseguramiento de calidad de software**, junto con el control de calidad del proceso y del producto.
- Luego de desarrollar software, las actividades de testing permiten revelar la calidad del software en sí, es decir, si el producto satisface los requerimientos definidos por el cliente.
- Hacer aseguramiento de calidad ayuda a que el producto llegue mejor al testing.

No confundir

- El testing no asegura la calidad por sí solo.
- El testing debe estar acompañado de otras actividades de calidad.
- Aseguramiento de calidad no es lo mismo que testing.
- El testing controla la calidad, pero no la asegura.

¿Qué es el testing?

El testing es un **proceso destructivo** que intenta encontrar defectos en el software.

No busca demostrar que el software funciona, sino encontrar defectos. Esto se debe a que se parte de la idea de que siempre pueden existir defectos, ya que el software es construido por personas.

Por eso, se comienza con la hipótesis de que el software es incorrecto. Un test es considerado exitoso cuando encuentra defectos.

Si un software tiene alta calidad, puede ser más difícil encontrar defectos. En cambio, no encontrar defectos no siempre significa que el software sea bueno; muchas veces puede significar que el testing fue insuficiente.

Si el software tiene bajo nivel de calidad, el costo de retrabajo será alto, porque habrá muchos ida y vuelta entre testing y desarrollo para corregir errores.

El testing suele representar entre el **30% y el 50% del costo total del proyecto**, por lo que se debe buscar un nivel óptimo de cobertura según costo-beneficio.

Además:

- Nunca puede testearse el 100% del software.
- Siempre hay que decidir qué pruebas hacer.
- Busca controlar la calidad.
- Un error en el software no puede atribuirse únicamente al testing, porque el testing no introduce defectos, sino que los visibiliza.

Una vez definidos los requerimientos de calidad

Se debe tener en cuenta que:

- La calidad no puede “inyectarse” al final.
- La calidad del producto depende de tareas realizadas durante todo el proceso.
- Detectar errores de forma temprana ahorra esfuerzo, tiempo y recursos.
- La calidad abarca tanto aspectos del producto como del proceso.
- Mejorar el proceso ayuda a evitar defectos recurrentes.
- El testing no puede asegurar por sí solo la calidad del software.

Error vs. defecto

La diferencia entre **error** y **defecto** está en el momento en el que se introducen y detectan.

El **error** se descubre a partir de técnicas específicas que me permiten encontrar los mismos dentro de la misma etapa en la que estoy trabajando

Los **defectos** nos demuestran un error no detectado que se trasladó a una etapa siguiente

Cuando hablamos de defectos encontrados durante el testing, debemos considerar dos aspectos que nos van a definir el accionar que vamos a realizar sobre estos:

SEVERIDAD	PRIORIDAD
La severidad se relaciona con la gravedad técnica del defecto. La determina la persona que realiza el testing. Puede ser: ○ Bloqueante: no permite continuar con el caso de prueba. ○ Crítico: compromete la ejecución del caso de prueba, aunque el sistema funciona. ○ Mayor: la funcionalidad testada funciona, pero de forma incorrecta. ○ Menor: la funcionalidad se ejecuta correctamente, pero con advertencias o errores de baja importancia. ○ Cosmético: se relaciona con cuestiones visuales, como fechas, números, interfaz gráfica, etc.	La prioridad se relaciona con la urgencia para resolver el defecto según las necesidades del cliente. La define el cliente y puede ser: ○ Alta. ○ Media. ○ Baja. La severidad y la prioridad son independientes entre sí. Por ejemplo, un defecto cosmético puede tener baja prioridad en un sistema de inventarios, pero alta prioridad en un sistema de una empresa de marketing.

Niveles de testing o prueba

Los niveles de testing determinan la granularidad de la prueba que se va a ejecutar.

Por lo general, se comienza probando componentes pequeños y luego se avanza hacia pruebas de mayor granularidad.

1. Pruebas unitarias

Se realizan sobre un componente individual de forma independiente, luego de construirlo. Son pruebas sobre aspectos puntuales y aislados del proyecto.

Características:

- Las realizan los mismos desarrolladores.
- Se incluyen dentro del Definition of Done.
- Encuentran errores, no defectos.
- Los errores se solucionan rápidamente.
- No suelen registrarse formalmente.
- Son fáciles de automatizar.
- Pueden usar herramientas específicas.
- Tienen un workflow de implementación.

2. Pruebas de integración

Buscan verificar si las partes que funcionan correctamente de forma aislada también funcionan bien en conjunto.

Para eso, se integran componentes que ya fueron probados en las pruebas unitarias.

Deben realizarse mediante pruebas de integración incrementales, porque permiten identificar mejor los errores.

Técnicas posibles:

- Integración de arriba hacia abajo, o top-down.
- Integración de abajo hacia arriba, o bottom-up.

Lo ideal es combinar ambas técnicas.

Puntos clave:

- Conectar de a poco.
- Probar tempranamente los módulos críticos.
- Minimizar la necesidad de programas auxiliares.

3. Pruebas de sistema o pruebas de versión → Son pruebas más amplias que las anteriores.

No prueban solo la integración de dos componentes, sino el sistema entero. Es decir, verifican que la aplicación funcione como un todo.

Buscan asegurar que el sistema completo funcione satisfactoriamente.

Incluyen aspectos como:

- Estrés.
- Performance.
- Recuperación ante fallas.
- Seguridad.
- Requerimientos funcionales.
- Requerimientos no funcionales.

El entorno de prueba debe parecerse lo más posible al entorno de producción para reducir riesgos.

Idealmente, estas pruebas deberían realizarlas personas ajenas al desarrollo del programa.

4. Pruebas de aceptación

Por lo general, las ejecuta el usuario final o cliente en la etapa de despliegue.

El foco no es encontrar defectos, sino generar confianza y familiaridad con el sistema, además de verificar que cumpla con lo pedido por el cliente.

El ambiente debe ser lo más parecido posible al entorno de producción.

Incluye:

- **Pruebas alfa:** realizadas por el usuario en ambiente de laboratorio.
- **Pruebas beta:** realizadas en ambientes reales de trabajo.

Según el enfoque, participan:

- **Tradicional:** usuario, gerente de testing, líder de proyecto y empleados funcionales.
- **Ágil:** usuario y todos los miembros del equipo, generalmente en la sprint review.

Ambientes para construcción de software

Los ambientes son los lugares o entornos donde se trabaja durante el desarrollo de software.

Cada ambiente tiene un propósito específico y se utiliza en distintas etapas del proceso de desarrollo y despliegue.

Es conveniente separar desarrollo de testing.

Ambiente de desarrollo

Es el lugar donde los desarrolladores crean, prueban y depuran el software.

Puede ser un entorno local o en red.

Características:

- Se utiliza para escribir y probar código antes de integrarlo al resto del sistema.
- Los programadores usan herramientas como librerías, compiladores y extensiones.
- Es flexible.
- Permite experimentar sin afectar al resto del sistema.
- Allí se realizan las pruebas unitarias.

Ambiente de prueba

Es el ambiente utilizado por los testers para realizar pruebas exhaustivas.

Su objetivo es verificar que el software funcione correctamente y cumpla con los requisitos.

Características:

- Los desarrolladores no deberían tener acceso.
- Suele tener características similares a producción, aunque no todas porque sería muy costoso.
- Allí se realizan pruebas de integración y pruebas de sistema.

Ambiente de preproducción

En este ambiente se realizan pruebas antes de lanzar el software a producción.

Características:

- Simula condiciones de producción.
- Permite hacer pruebas de carga, estrés, seguridad, entre otras.
- Busca validar y verificar que el software esté listo para producción.
- Debería parecerse lo máximo posible al ambiente productivo.
- Allí se realizan las pruebas de aceptación.

Ambiente de producción

Es el lugar donde el software está funcionando y es accesible a los usuarios finales.

Características:

- Es el entorno real de uso.
- Debe ser seguro, escalable y confiable.
- Debe tener medidas de recuperación y respaldo.
- Debe garantizar disponibilidad continua.
- No es un lugar para realizar pruebas.

Casos de prueba → El caso de prueba es el artefacto más importante del testing.

Tiene que ver con un conjunto de condiciones o variables que nos van a permitir determinar si el software está funcionando correctamente o no. La buena definición de casos de prueba nos ayuda a reproducir defectos.

Consiste en una secuencia de pasos que describe acciones a realizar para lograr un resultado concreto y esperado.

Debe especificar claramente:

- Qué acciones se ejecutan.
- Qué datos se usan.
- Qué resultado se espera.

Un caso de prueba es un conjunto de condiciones o variables bajo las cuales un tester determina si el software funciona correctamente o no.

La buena definición de casos de prueba ayuda a reproducir defectos y solucionarlos, porque permite conocer la secuencia de pasos y las variables usadas para encontrar el defecto.

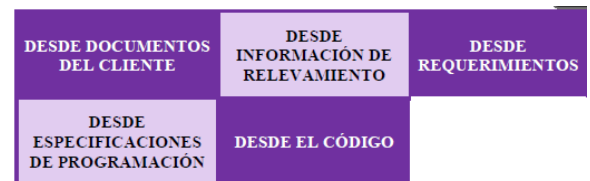
Al definir casos de prueba, se intenta apuntar a valores límites o esquinas, porque allí suelen esconderse muchos defectos.

Los casos de prueba buscan descubrir la mayor cantidad de errores con el mínimo esfuerzo y tiempo posible.

También permiten validar y verificar.

Mientras no cambien los requerimientos, pueden reutilizarse las veces que sea necesario. Si no están actualizados, no se pueden ejecutar correctamente.

Los casos de prueba pueden derivar de distintos lugares, **cuanta más documentación tengamos, más fácil será especificar casos de prueba**. Cuanta menos especificación tengamos, vamos a tener que describir con mayor detalle los casos de prueba



Los requerimientos son la fuente más importante porque permiten validar y verificar.

El código permite verificar, pero no necesariamente validar, ya que no asegura que sea lo que el cliente quiere.

El éxito de un buen testing depende de qué tan bien diseñados estén los casos de prueba y qué tan bien se hayan preparado las bases de datos para las pruebas.

Componentes de un caso de prueba

Un caso de prueba está compuesto por:

- **Objetivo:** característica del sistema que se quiere comprobar.
- **Datos de entrada y ambiente:** datos que se introducen en el sistema bajo ciertas condiciones preestablecidas.
- **Comportamiento esperado:** salida o acción esperada del sistema según sus requerimientos.

Condiciones de prueba

- Una condición de prueba es la reacción esperada de un sistema frente a un estímulo particular.
- Ese estímulo está formado por distintas entradas concretas.
- Primero se definen las condiciones de prueba y luego los casos de prueba.
- Cada condición debe probarse en al menos un caso de prueba.

Estrategias y métodos de prueba

El objetivo es diseñar la menor cantidad posible de casos de prueba que permitan cubrir la mayor cantidad de funcionalidades y escenarios, optimizando tiempo, esfuerzo y costos.

Por eso se utilizan técnicas que buscan maximizar la efectividad de las pruebas con la menor cantidad de casos posibles. No existe un único método perfecto. Lo mejor es combinar distintas técnicas.

CAJA BLANCA	CAJA NEGRA
A diferencia de caja negra, lo que hace caja blanca es mirar dentro del código, revisar su estructura y ver si dentro de esa definición se encuentra algo que vaya a dar un resultado no esperado. Se basan en el análisis de la estructura interna del software o un componente del software. Hay distintos métodos. Algunos de ellos son: - COBERTURA DE ENUNCIADOS O CAMINOS BASICOS - COBERTURA DE SENTENCIAS - COBERTURA DE DECISIÓN - COBERTURA DE CONDICIÓN	Se llama de caja negra, ya que nosotros definimos en los casos de prueba cuales son las entradas y el resultado esperado tiene que ver con cómo se comporta el sistema frente a esas entradas, lo que nosotros desconocemos, es específicamente como se llega a esa salida. Frente a determinadas entradas esperamos obtener ciertas salidas, si las obtenemos el caso de prueba pasa. Los métodos de caja negra pueden dividirse en dos. - BASADOS EN ESPECIFICACIONES Algunos métodos son: • Partición de Equivalencias • Análisis de valores límites - BASADO EN LA EXPERIENCIA Tiene que ver con el testing que hace el tester experimentado y sabe exactamente dónde ir para

Estrategia de caja negra

- La caja negra es una estrategia dinámica, porque se ejecuta el código.
- No se analiza la estructura interna de la implementación. Se observan las funcionalidades en términos de entradas y salidas.
- Se ingresan datos a una funcionalidad y luego se comparan los resultados obtenidos con los resultados esperados.

Caja negra basada en especificaciones

- Se ejecuta utilizando documentación de especificaciones del producto.

Partición de equivalencias

- Analiza las condiciones externas de una funcionalidad, como entradas y salidas.
- Luego divide esas condiciones en clases de equivalencia, es decir, subconjuntos de valores posibles que producen resultados equivalentes.

Ejemplos de entradas:

- Campos de texto.
- Archivos.

Ejemplos de salidas:

- Mensajes en pantalla.
- Tablas.

La idea principal es que, si todos los valores de un grupo producen un resultado equivalente, alcanza con probar uno de ellos para representar a toda la clase.

Pasos:

1. Identificar clases de equivalencia válidas y no válidas.
2. Verificar que no falte ningún subconjunto.
3. Recordar que la unión de todos los subconjuntos debe cubrir todo el universo de la condición externa.
4. Identificar casos de prueba.

Ejemplo de partición de equivalencias

Supongamos un sistema que valida la edad de una persona para permitir el ingreso a una actividad.

La condición de entrada es la edad y el sistema acepta únicamente edades entre 18 y 100 años.

Clase válida:

- Números enteros entre 18 y 100.

Clases inválidas:

- Números enteros menores a 18.
- Números enteros mayores a 100.
- Números negativos.
- Números no enteros.
- Caracteres no numéricos.

Cada clase agrupa valores que generan un comportamiento equivalente.

Por ejemplo, cualquier edad menor a 18 produce el mismo resultado: el sistema rechaza el acceso.

Análisis de valores límites

- Es una variante de la partición de equivalencias.
- En vez de seleccionar cualquier elemento representativo de una clase, se seleccionan los bordes de la clase para definir los casos de prueba.
- La mayor cantidad de defectos suele aparecer en los extremos de las clases de equivalencia.
- Por ejemplo, si el rango válido es de 18 a 100, conviene probar valores como: 17/18/100/101

Caja negra basada en la experiencia

- Se basa en la experiencia y el conocimiento del tester.
- El tester sabe hacia dónde ir para encontrar defectos gracias a su experiencia.

Adivinanza de defectos

Consiste en probar situaciones donde generalmente falla el software.

Por ejemplo:

- Base de datos vacía.
- Base de datos llena.
- Acciones sin permisos.
- Datos inconsistentes.

Testing exploratorio

- El tester aprende a manejar el sistema mientras lo prueba.
- A partir de su experiencia y creatividad, genera nuevas pruebas para lograr una mejor cobertura.

Estrategia de caja blanca

La caja blanca se basa en el análisis de la estructura interna del software o de un componente.

Se mira el código para analizar aspectos como:

- Complejidad.
- Uso de palabras reservadas.
- Errores de sintaxis.
- Ciclos sin fin.
- Loops infinitos.

Limitaciones de la caja blanca

Tiene dos grandes limitaciones:

- El número de caminos lógicos únicos puede ser muy grande.
- Las pruebas exhaustivas de caminos no aseguran que el programa sea correcto respecto de las especificaciones.

Además, no siempre detecta caminos faltantes ni errores de datos sensibles.

Tipos de cobertura

- **Cobertura de sentencias** → Busca ejecutar al menos una vez cada instrucción del código. No importa evaluar todos los resultados de las condiciones.
- **Cobertura de decisión:** Busca probar que cada decisión del código tome al menos una vez la rama verdadera y la rama falsa.
- **Cobertura de condición:** Busca que cada condición individual dentro de una decisión sea evaluada al menos una vez como verdadera y una vez como falsa.
No importa qué resultado tome la decisión completa.
- **Cobertura de decisión/condición:** Busca evaluar tanto las ramas verdadera y falsa de cada decisión como los valores verdadero y falso de cada condición individual. Combina cobertura de decisión y cobertura de condición.
- **Cobertura múltiple:** Busca probar todas las combinaciones posibles de valores verdaderos y falsos de las condiciones. Es el nivel más exhaustivo sobre las decisiones del código.
- **Cobertura de caminos básicos:** Busca identificar y ejecutar todos los caminos independientes de una funcionalidad al menos una vez.
Utiliza un grafo de flujo y se calcula la complejidad ciclomática, que indica la cantidad mínima de casos de prueba necesarios para recorrer todos los caminos básicos del código.
- **Ciclos de prueba o ciclo de Test :** Un ciclo de pruebas abarca la ejecución de la totalidad de los casos de prueba establecidos aplicados a una misma versión del sistema a probar; ejecuto todos los casos de prueba, veo cuales fallaron, corrijo los defectos, y hago otro ciclo, lo que implica correr todos los casos de prueba en la nueva versión que ya tiene los defectos corregidos, esto en loop hasta que cumplamos con el criterio de aceptación que hayamos definido para el Testing.
Con el cliente debemos definir en qué momento vamos a dejar de probar; es imposible probar hasta que no haya defectos.
Si aparecen defectos durante la ejecución, no se interrumpe la corrida, salvo que sea un defecto invalidante. Por ejemplo, si no se puede iniciar sesión y eso impide continuar.
- **Ciclos de prueba con regresión:** Al concluir un ciclo de pruebas, y reemplazarse la versión del sistema sometido al mismo, debe realizarse una verificación total de la nueva versión, a fin de prevenir la introducción de nuevos defectos al intentar solucionar los detectados.
La teoría nos dice que cuando solucionamos un defecto normalmente se introducen 2/3 defectos más. Es importante volver a verificar la nueva versión en su totalidad para asegurarnos de no haber introducido nuevos defectos. No siempre debemos hacer regresión, eso lo vamos a ir viendo a medida que testearmos.
- **Sin regresión** En cada ciclo se prueba solo lo que falló anteriormente para verificar si fue corregido. Esto agiliza el proceso, pero tiene riesgo porque al corregir un defecto pueden introducirse otros defectos nuevos.

Proceso de testing en ambientes tradicionales

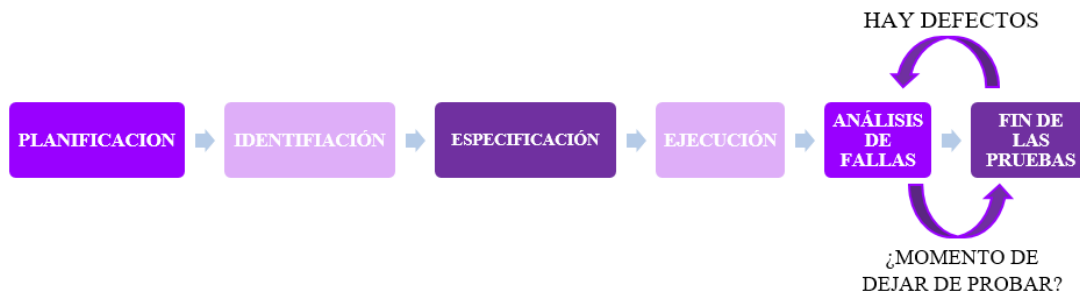
- El testing es un proceso definido que se lleva a cabo durante todo el ciclo de vida del producto.

Testing ad hoc

- El testing ad hoc se realiza sin un proceso definido.
- Se prueba libremente, sin registrar paso a paso lo que se hizo.
- Esto hace que se pierda trazabilidad.

Etapas del proceso de testing

En el contexto de un proceso definido de testing, es el siguiente:



PLANIFICACION: Se trata de la parte más estratégica del testing, se genera un Plan de Prueba el cual debe contener

- Riesgos y objetivos del testing.
- Estrategia de testing.
- Recursos.
- Criterios de aceptación.

IDENTIFICACION Y ESPECIFICACION: Se definen y escriben los Casos de Prueba - Se priorizan los casos según lo definido en la planificación Y Se diseña el entorno de prueba.

EJECUCION: Se lleva a cabo la ejecución de las pruebas (de manera manual o automatizada, o una combinación de ambas)

- Se registran los resultados reales.
- Se comparan con los resultados esperados.
- Se genera un reporte de defectos.
- Se indican defectos encontrados, condiciones y entornos.

ANALISIS DE FALLAS: Se analiza si la solución/resultados de la ejecución de los casos de prueba fueron o no los esperados. Si obtuvimos los resultados esperados, ocurre el **FIN DE LAS PRUEBAS**, si no cumplió con lo esperado voy a hacer un análisis de fallas para corregirlo.

Este ciclo entre ejecución y análisis de fallas se ejecuta tantas veces como sea necesario para llegar al criterio de aceptación que va a determinar luego el

FIN DE LAS PRUEBAS. Se verifican las metas y objetivos del cliente, los stakeholders, el proyecto y los riesgos de las pruebas. En organizaciones más maduras, se deja de probar cuando no hay defectos bloqueantes, críticos ni mayores, y cuando los defectos menores o cosméticos son pocos. Estos defectos menores pueden quedar documentados en la nota de release. En esta etapa puede confeccionarse un informe final.

El ciclo entre ejecución y análisis de fallas se repite todas las veces necesarias hasta alcanzar el criterio de aceptación.

Entregables o artefactos de testing

- El proceso de testing genera distintos entregables:

Plan de pruebas → Es análogo al plan de proyecto → Define la planificación general del testing.

Casos de prueba → Describen las pruebas que se van a ejecutar, con sus pasos, datos y resultados esperados.

Reporte de incidentes o defectos

Luego de ejecutar los casos de prueba, se elabora un reporte indicando:

- Qué casos pasaron.
- Qué casos fallaron.
- Qué defectos se encontraron.
- Cómo reproducir esos defectos.

Esto permite que los desarrolladores corrijan los defectos y envíen nuevamente el producto a testing.

Informe final

Suele contener métricas e información final del incremento del producto.

Puede incluir:

- Cantidad de ciclos.
- Cantidad de errores por ciclo.
- Métodos utilizados.
- Tipos de prueba aplicados.

Este es el único artefacto que puede ser negociable en algunas organizaciones informales.

También tiene utilidad estadística.

VERIFICACIÓN	VALIDACIÓN
La verificación apunta a ver si estamos construyendo el sistema correctamente . Libre de defectos, apuntando a la perfección de su funcionalidad.	La validación apunta a ver si estamos construyendo el sistema correcto , es decir, el sistema que cumple efectivamente con los requerimientos del cliente.

El testing y el ciclo de vida del software

- Es importante involucrar las actividades de testing lo más temprano posible en el ciclo de vida del software.
- Si ya están definidos los requisitos, se pueden empezar a definir los casos de prueba, incluso sin tener código escrito.
- Para comenzar a hacer testing se necesitan:

Cero líneas de código.

Empezar temprano permite:

- Dar visibilidad al equipo sobre cómo se va a probar el producto.
- Disminuir costos de corrección.
- Encontrar errores antes de que se conviertan en defectos.

¿Cuánto testing es suficiente? Es IMPOSIBLE probar todas las opciones posibles.

El testing exhaustivo no es viable.

Decidir cuánto testing es suficiente depende de:

- Nivel de riesgo.
- Costos del proyecto.
- Criterios de aceptación.
- Confianza alcanzada sobre el funcionamiento del sistema.

Los riesgos ayudan a priorizar:

- Qué se testea primero.
- Con qué esfuerzo se testea.
- Qué se deja para después.

El testing puede finalizar cuando existe suficiente confianza en que el sistema funciona correctamente.

Criterios posibles para finalizar el testing

Se puede definir el fin del testing según:

- Costo o momento específico.
- Porcentaje de tests ejecutados sin fallas.
- Cantidad aceptable de fallas no críticas.
- Defectos encontrados similares a los defectos estimados.
- Ejecución exitosa del conjunto de pruebas diseñado.
- Cobertura estructural alcanzada.
- Inexistencia de defectos de determinada severidad.
- Fallas predichas que aún se mantienen en el software.

Malas concepciones sobre cuándo terminar

No es correcto decir que:

- El testing nunca es suficiente.
- Se termina cuando el sistema funciona correctamente.
- Se termina cuando se ejecutó todo lo planificado.

Mitos del testing

Son mitos o ideas incorrectas:

- El testing comienza recién al terminar de codificar.
- El testing consiste en probar que el software funciona correctamente.
- Testing es igual a calidad del producto o del proceso.
- El tester es enemigo del programador.
- El testing muestra que no hay errores.
- El testing demuestra que un programa hace lo que debe hacer.

Lo correcto es que el testing busca agregar valor al producto revelando su calidad y brindando confianza en el software.

No se prueba un sistema para demostrar que funciona, sino que se parte de la suposición de que existen defectos y se intenta encontrar la mayor cantidad posible.

Un programa puede hacer lo que se supone que debe hacer y, aun así, contener defectos.

Tipos de pruebas

Smoke Test	Sanity Test					
Es la primera corrida de los test de sistema en la cual se verifica que las funcionalidades básicas de una aplicación o sistema funcionen correctamente antes de realizar pruebas más exhaustivas. • Es una corrida rápida. • Busca detectar fallas de gran magnitud. • Se realiza antes del ciclo 0. • Evita comenzar el testing formal si todavía hay errores graves.	Se realiza después de una ronda de pruebas más exhaustivas para verificar si se han corregido los errores o problemas críticos identificados previamente. El objetivo principal del sanity test es asegurarse de que las correcciones realizadas no hayan introducido nuevos problemas y que las funcionalidades clave del software sigan funcionando correctamente después de las correcciones					
Testing funcional - ¿Qué hace el sistema?	Testing no funcional - ¿Cómo funciona el sistema? ¿Cómo lo hace?					
Las pruebas se basan en funciones y características (descriptas en los documentos de requerimientos y entendidas por los testers) del software y su interoperabilidad con sistemas específicos. Si los requerimientos están bien planteados el caso de prueba deriva directamente de ellos • Basado en requerimientos → se prueban REQ específicos enfocados en funcionalidad • Basado en los procesos de negocio → se prueba un proceso completo de negocio	Las pruebas se basan en aspectos no relacionados directamente con las funciones específicas de un software. Los REQ funcionales igual de importantes que los REQ no F. Estas pruebas se centran en características como: • Rendimiento. • Seguridad. • Usabilidad. • Escalabilidad. Tipos de testing no funcional <table><tr><td>Performance Testing</td><td rowspan="4">Pruebas de Mantenimiento Pruebas de Fiabilidad Pruebas de Portabilidad</td></tr><tr><td>Pruebas de Carga</td></tr><tr><td>Pruebas de Stress</td></tr><tr><td>Pruebas de Usabilidad</td></tr></table>	Performance Testing	Pruebas de Mantenimiento Pruebas de Fiabilidad Pruebas de Portabilidad	Pruebas de Carga	Pruebas de Stress	Pruebas de Usabilidad
Performance Testing	Pruebas de Mantenimiento Pruebas de Fiabilidad Pruebas de Portabilidad					
Pruebas de Carga						
Pruebas de Stress						
Pruebas de Usabilidad						

Modelo en V

- El modelo en V se utiliza para determinar cuándo se está en condiciones de realizar ciertas actividades según la etapa de desarrollo del software.
- En el desarrollo se avanza de lo general a lo particular.
- Es decir, se parte de requerimientos de granularidad gruesa y se avanza hacia componentes de granularidad fina, como clases o módulos.
- En las pruebas se realiza el camino inverso: se va de lo particular a lo general.
- Es decir, se prueba en sentido inverso a cómo se desarrolló.

¿Por qué el testing es necesario?

Razones correctas

El testing es necesario porque:

- La existencia de defectos en el software es inevitable.
- Ayuda a reducir riesgos.
- Construye confianza en el producto.
- Las fallas pueden ser muy costosas.
- Permite verificar que el software se ajusta a los requerimientos.
- Permite validar que las funciones se implementan correctamente.
- Puede evitar problemas legales con el cliente.

Razones incorrectas

No es correcto justificar el testing diciendo que:

- Sirve para llenar el tiempo entre el fin del desarrollo y el release.
- Sirve para probar que no hay defectos.
- Se hace solo porque está incluido en el plan del proyecto.
- Se hace porque debuggear el código es rápido y simple.

Principios del testing

- 1) El testing muestra la presencia de defectos, no su ausencia.
- 2) Se debe testear de forma temprana.
- 3) El testing exhaustivo es imposible.
- 4) El testing depende del contexto.
- 5) Es una falacia decir que al terminar el testing hay ausencia total de errores.
- 6) Los defectos suelen agruparse en pocas áreas del sistema.
- 7) La repetición de los mismos casos puede generar la paradoja del pesticida.
- 8) Un programador debería evitar probar su propio código, salvo en pruebas unitarias.
- 9) No se prueba solo que el sistema haga lo que debe hacer, sino también qué ocurre si se hace lo que no debe hacerse.
- 10) No se debe planificar el testing suponiendo que no se encontrarán defectos.
- 11) El objetivo no es probar que el software funciona, sino encontrar defectos.

Agrupamiento de defectos

- Los defectos suelen concentrarse en un conjunto limitado de áreas.
- Se suele decir que el 80% de los defectos se agrupan en el 20% de la funcionalidad.
- Por eso, las pruebas deben priorizarse y enfocarse en esas zonas más críticas.

Paradoja del pesticida

Cuando se ejecutan muchas veces los mismos casos de prueba, puede ocurrir que esos casos ya no encuentren fallas nuevas. Los verdaderos defectos pueden quedar ocultos si no se actualizan o varían las pruebas.

El programador y su propio código → Un programador debería evitar probar su propio código, salvo en pruebas unitarias.

- Puede perderse el carácter destructivo del testing.
- El programador puede no querer encontrar sus propios defectos.
- Puede haber errores por malentendidos del dominio.
- Si él mismo testea, esos defectos pueden no detectarse.

Testing en Ágil

Valores del testing ágil

- El testing ágil surge alineado con el **Manifiesto Ágil** y plantea las bases para construir una forma de testing integrada al desarrollo.
- El primer aspecto que aparece en el manifiesto habla de que NO se hace testing al final, sino que se hace mientras se va construyendo el software. Es responsabilidad de todo el equipo, no necesariamente hay un tester con un rol determinado, sino que cualquiera puede tomar ese rol.
- El manifiesto plantea además todo lo que se tiene en cuenta en el testing en ambientes ágiles. Prevenir bugs sobre encontrar bugs, es decir, prevenir los errores, no encontrar directamente defectos.
- Darle lógica a lo que estamos testeando, no querer solo tildar una casilla y verificar una funcionalidad. El tester debe querer encontrar errores/defectos del sistema para construir un mejor sistema, no para romperlo.

Valores principales

1. Pruebas durante el desarrollo sobre pruebas al final

- En testing ágil, las pruebas no se realizan únicamente al final del desarrollo.
- El testing se hace durante todo el proceso, acompañando la construcción del software.
- Por eso, el testing es una **actividad continua**, no una fase separada.

2. Prevenir defectos sobre encontrar defectos

- En el enfoque tradicional, el testing busca encontrar defectos.
- En cambio, en el enfoque ágil se busca principalmente **prevenirlos**.
- Para eso, se mejora el proceso de desarrollo con el objetivo de reducir la aparición de errores.

3. Entender lo que está pasando sobre verificar la funcionalidad

- No alcanza solamente con comprobar que algo funciona.
- También es importante comprender **cómo funciona y por qué funciona** de esa manera.

4. Construir un mejor sistema sobre romper el sistema

- El testing ágil no busca “romper” el sistema, sino colaborar para mejorar la calidad del producto.

5. Responsabilidad de todo el equipo por la calidad sobre el tester como único responsable

- En ágil, la calidad no es responsabilidad exclusiva del tester o del equipo de QA.
- La calidad es responsabilidad de **todo el equipo**.

Testing tradicional	Testing ágil
Modelo secuencial por etapas separadas: requerimientos, diseño, programación, testing y despliegue.	Ciclo continuo e iterativo.
Cada fase depende de que termine la anterior.	Las actividades se realizan de manera conjunta y constante.
Los cambios suelen ser lentos y costosos.	Permite corregir errores temprano y adaptarse mejor al cambio.
Encontrar errores en etapas avanzadas resulta costoso.	Permite entregar funcionalidades más rápido.
Prioriza la planificación y la secuencialidad.	Prioriza la retroalimentación, la flexibilidad y la entrega continua de valor.

Principios del testing ágil

1. **EL TESTING SE MUEVE HACIA ADELANTE EN EL PROYECTO:** No se realiza al final del proyecto, sino que se va haciendo de manera constante al finalizar cada iteración.
2. **TESTING NO ES UNA FASE:** Se hace mientras se desarrolla el código
3. **TODOS HACEN TESTING:** No hay un rol específico de tester, cualquiera dentro del proyecto puede testear.
4. **REDUCIR LA LATENCIA DEL FEEDBACK**
5. **LAS PRUEBAS REPRESENTAN EXPECTATIVAS**
6. **MANTENER EL CODIGO LIMPIO, CORREGIR LOS DEFECTOS RAPIDO:** Cuando hablamos del manifiesto hablamos de prevenir errores antes de solucionarlos
7. **REDUCIR LA SOBRECARGA DE DOCUMENTACION EN LAS PRUEBAS**
8. **LAS PRUEBAS SON PARTE DEL “DONE”:** En el criterio de DONE se incluye la realización de pruebas.
9. **DE PROBAR AL FINAL A “CONDUcido POR PRUEBAS”**

Prácticas concretas del testing ágil

- **TESTING UNITARIO O DE INTEGRACIÓN AUTOMATIZADO:** Todo lo que yo ejecuto de manera mecánica se automatiza. Gano tiempo, ahorro recursos y los utilizo en otro contexto.
- **M TESTING REGRESIVO A NIVEL DE SISTEMA AUTOMATIZADAS:** Las pruebas de regresión también podrían automatizarse.
- **TESTING EXPLORATORIO:** El testing exploratorio no se puede automatizar
- **TDD (Test Driven Development)**
- **ATDD (Desarrollo conducido por Pruebas de Aceptación):** Esta relacionado con TDD, pero tiene que ver con el desarrollo conducido por las pruebas de aceptación.
- **CONTROL DE VERSIÓN DE LAS PRUEBAS CON EL CÓDIGO:** Junto con el código tengo asociados los **Unit Tests** y los **System Tests** que tengo que correr y de esa manera es fácil plantear la automatización del testing

Automatización

La automatización aporta valor porque permite ejecutar pruebas de forma rápida, repetible y confiable.

Beneficios de automatizar

- Rápida ejecución.
- Reusabilidad.
- Mayor cobertura.
- Mayor precisión.
- Mayor alcance.
- Reducción de costos.
- Feedback rápido.

La automatización se logra mediante herramientas que permiten ejecutar pruebas automáticamente.

Debe implementarse gradualmente, comenzando por los tests más críticos o repetitivos.

Luego se automatizan pruebas simples y se amplía la cobertura iteración tras iteración.

Pruebas que se pueden automatizar

Se pueden automatizar:

- Pruebas unitarias.
- Pruebas de integración.
- Pruebas de regresión.

Roles y competencias del tester

Tester en ambiente tradicional	Tester en ambiente ágil
El tester suele trabajar al final del desarrollo, cuando el sistema ya fue construido. Su tarea principal es encontrar errores de forma destructiva, partiendo de la idea de que existen defectos. Tareas del tester tradicional • Diseñar casos de prueba. • Ejecutar casos de prueba. • Detectar bugs. • Reportar bugs. • Validar y verificar. • Documentar resultados	El tester es un rol, no necesariamente una persona específica. Todos los miembros del equipo participan de la calidad. El tester trabaja durante todo el Sprint de forma colaborativa con: • Developers. • Product Owner. • System Team. El objetivo principal es evitar que aparezcan errores. Tareas del tester en ágil • Validar el “Done” de las historias. • Registrar y comunicar bugs. • Analizar métricas de calidad. • Acompañar la aceptación de la historia junto con el Product Owner. • Preparar y mantener el ambiente de pruebas. • Preparar casos y datos de prueba para las historias del Sprint.

Gestión del cambio

El cambio hacia Agile no ocurre de manera inmediata y suele generar resistencia.

Algunas personas pueden sentirse incómodas por:

- Nuevos roles.
- Pérdida de costumbres.
- Incertidumbre sobre la forma de trabajo.

Para introducir el cambio se recomienda:

- Tener paciencia.
- Acompañar el proceso.
- Hablar abiertamente sobre los miedos del equipo.
- Dar participación a los miembros.
- Celebrar logros y avances.
- Capacitar continuamente.

Manejo de expectativas

Muchas veces la gerencia espera resultados rápidos al adoptar Agile, pero el cambio cultural lleva tiempo.

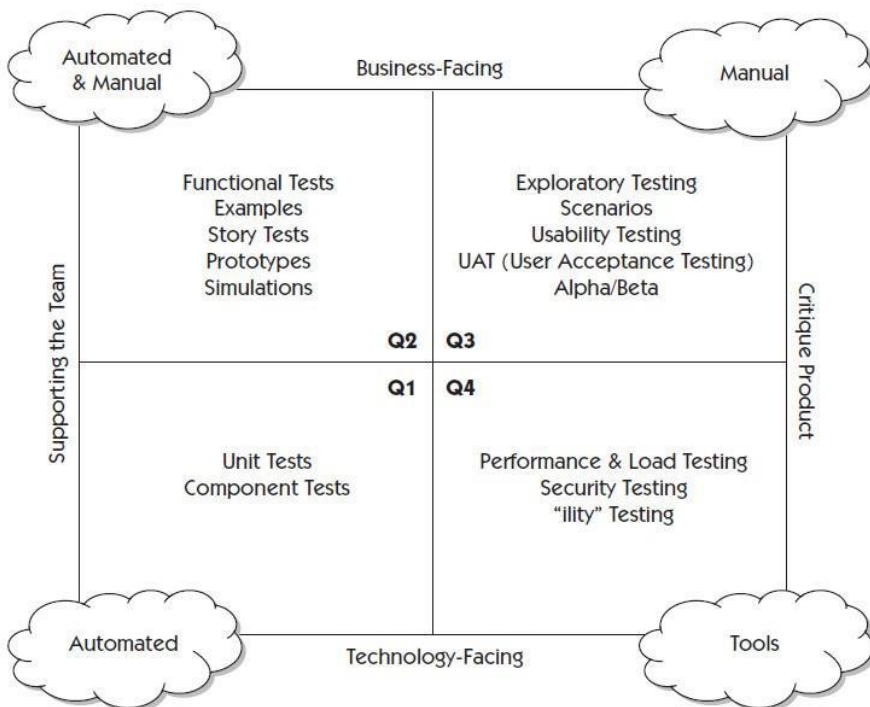
También pueden existir diferencias entre lo que esperan:

- Managers.
- Testers.
- Developers.
- Clientes.

Para manejar expectativas es importante:

- Comunicar objetivos claros.
- Hablar el lenguaje del negocio y de la gestión.
- Alinear expectativas entre todas las áreas.
- Comprender que Agile busca mejora

Cuadrantes del testing ágil



Nos hablan de cuales son todos los tipos de testing disponibles, cual es el foco de cada uno, hacia donde apuntan y cuales son aquellos que pueden automatizarse y los que deben realizarse de manera manual.

Los cuadrantes 1 y 4 tienen que ver con aspectos tecnológicos del testing, y los 2 y 3 tienen más que ver con la parte del negocio del testing.

Específicamente el cuadrante 4 tiene que ver con aspectos no funcionales como tests de performance, seguridad, etc.

Los cuadrantes 1 y 2 tienen más relación con el soporte al equipo y los 3 y 4 están más orientados a lo que

hacemos sobre el producto.

Permiten analizar:

- Si apuntan más al negocio o a la tecnología.
- Si ayudan al equipo durante el desarrollo.

- Si evalúan el producto terminado.
- Si pueden automatizarse o no.

Clasificación general

- **Cuadrantes de negocio o funcionales:** Q2 y Q3.
- **Cuadrantes de tecnología o técnicos:** Q1 y Q4.
- **Cuadrantes de soporte al equipo, generalmente automatizados:** Q1 y Q2.
- **Cuadrantes de producto terminado, generalmente manuales:** Q3 y Q4.

Q1

- Verifica que el código funcione correctamente.
- Ayuda a detectar errores temprano.

Q2

- Verifica que las historias de usuario y los criterios de aceptación se cumplan correctamente.

Q3

- Busca aprender sobre el producto y evaluar la experiencia del usuario.

Q4

- Valida la estabilidad, el rendimiento y la confiabilidad del sistema ya terminado.

Pirámide ágil vs. tradicional

La diferencia principal está en dónde se concentra el esfuerzo de prueba y cuál es el objetivo del testing.

Pirámide tradicional

En el enfoque tradicional, la mayor parte de las pruebas se realiza en la capa superior.

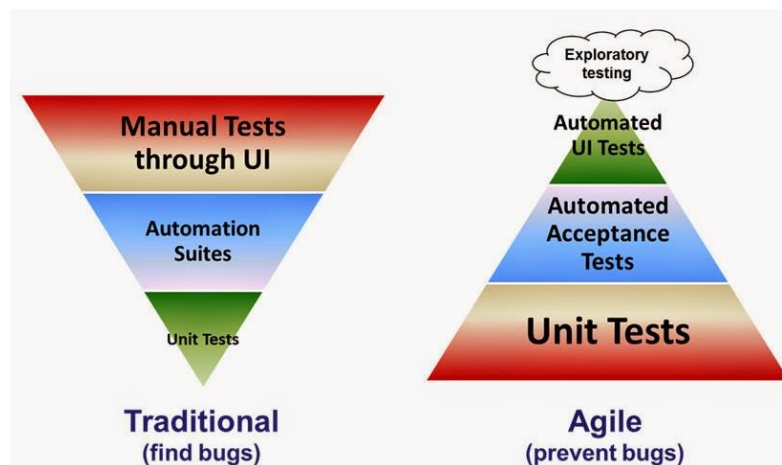
Se utilizan principalmente pruebas **End-to-End** o de interfaz gráfica, también llamadas pruebas GUI.

Estas pruebas buscan verificar el sistema completo.

Características

- Son más lentas.
- Son más costosas.
- Son difíciles de mantener.
- El testing ocurre al final del desarrollo.
- Se orienta a encontrar errores en etapas tardías.
- La mayoría de las pruebas son manuales.
- Hay poca automatización.

Pirámide ágil



En el enfoque ágil, se propone una base mucho más grande de pruebas unitarias o de componentes.

Estas pruebas se realizan en etapas tempranas y generalmente son hechas por los desarrolladores.

- **Base de la pirámide**

Incluye pruebas unitarias y pruebas de componentes.

Permiten validar pequeñas partes del código de manera rápida y automática.

Ayudan a detectar errores antes de que se propaguen.

- **Nivel intermedio**

Incluye pruebas funcionales, reglas de negocio y pruebas sobre la capa de servicio.

- **Parte superior**

Incluye pocas pruebas de flujo de trabajo mediante interfaz gráfica.

Estas pruebas se reducen porque son más lentas y menos prácticas de automatizar.

Objetivo del enfoque ágil

El testing ágil busca prevenir bugs mediante:

- Automatización temprana.
- Integración continua.
- Retroalimentación rápida.
- Pruebas de aceptación automatizadas.
- Pruebas unitarias automatizadas.

Se intenta reducir al mínimo las pruebas manuales y enfocarlas en casos difíciles de automatizar.

TDD

Que es

- Desarrollo guiado por pruebas de software, o Test-driven development (TDD)
- Es una técnica avanzada que involucra otras dos prácticas
 - Escribir las pruebas primero (Test First Development) (uso pruebas unitarias)
 - Refactorización (Refactoring)
- Práctica desarrollo de software que se enfoca en escribir pruebas antes de desarrollar código (escribo las pruebas que voy a realizar primero y luego hago el código necesario para hacer que esas pruebas pasen)
- Centrada en desarrollo funcional

Tradicional vs TDD

TRADICIONAL	TDD
Los desarrolladores diseñan las pruebas unitarias después de escribir el código	Los desarrolladores diseñan las pruebas unitarias antes de escribir el código
Aseguramiento y control de calidad más reactivo	Aseguramiento y control de calidad más proactivo

Pasos para realizar un TDD

- Escribir un test
 - Antes de escribir una línea de código tengo que pensar cómo voy a probar esa línea
 - Analizo los escenarios posibles y escribo un test que represente el comportamiento esperado del sistema antes de codear
- Hacer que el test compile
 - Una vez escrito el test, se agregan las clases, métodos o estructuras mínimas necesarias para que el código pueda compilar correctamente
 - En esta etapa todavía no importa si la funcionalidad funciona, sino que el entorno esté preparado para ejecutar la prueba.
- Hacer que el test falle (Red)
 - Se ejecuta el test y se verifica que falle, esto confirma que la funcionalidad aún no está implementada y que el test realmente está comprobando algo válido.
 - El estado “Red” representa justamente el fallo esperado de la prueba.
- Hacer un cambio para que pase el test
 - Se implementa el código mínimo necesario para que la prueba deje de fallar.
 - El objetivo no es crear una solución perfecta, sino una implementación simple que cumpla con lo que el test pide.
- Correr el test para que pase (Green)
 - Luego de realizar los cambios, se ejecutan nuevamente las pruebas para comprobar que ahora pasan correctamente.
 - El estado “Green” indica que la funcionalidad requerida ya cumple con las condiciones definidas en el test.
- Quitar duplicaciones (Refactor)
 - Se explica más abajo

Beneficios

- Obtenemos un código robusto, un código
 - Más predecible y limpio (permite saber cuándo el código está terminado)
 - Más tolerable al cambio, al tener escritas las pruebas es más fácil de entender
 - Más seguro
 - Aumenta la posibilidad de reducir la duplicación de código (si el test pasa el código ya existe)
 - Más barato de mantener, al ser más entendible
- Mayor velocidad de desarrollo y permite despliegue continuo
- Asegura cobertura de prueba, lo que conlleva a código de calidad
- Obliga a pensar en cómo usar el componente, por lo tanto, el diseño de las API
- Promueve el desarrollo de componentes con alta cohesión y bajo acoplamiento
- Si pienso primero en las pruebas reduzco errores

Leyes de TDD

- No escribir una línea de código hasta que no hayas escrito antes un test case que falla
- No es necesario escribir más test de los necesarios para que falle. Normalmente con un test que falle es suficiente.
- No vas a escribir más código en Producción que el necesario para que el test pase.

Patrones

PATRONES RED BAR	PATRONES GREEN BAR	PATONES TDD
OneStep Test Escribe un solo test que verifique una pieza mínima de funcionalidad.	Fakeit Devuelve un valor fijo o solución "falsa" solo para hacer pasar el test	Test Para nombrar las pruebas con la palabra test y la clase a probar
Starter Test Primer test que te ayuda a explorar la API o comportamiento deseado.	Triangulación Introduce un segundo test con diferentes datos para forzar una implementación más general	Isolatedtest Mantener los test totalmente independientes
Explanation test Escribe comentarios o preguntas para aclarar qué esperas que haga el código	Obvious implementation: Cuando la solución es trivial y obvia, puedes implementarla directamente sin "Fakeit"	Test List Primero escribe la lista de tests que vas a realizar
Learning test Permite conocer APIs externas sobre las que no sabemos su comportamiento	One to many Patrón para resolver test para colecciones de elementos.	Test first Escribe primero la prueba
Another Test Añade un nuevo test para cubrir un caso adicional o una variante del comportamiento		Asserttest primero escribir la confirmación y luego la prueba
RegressionTest Agrega un test que reproduce un bug descubierto, para evitar regresiones futuras		Test data Usa datos para el testing que sean fáciles de leer y seguir.
		Evidentdata Incluye los resultados esperados y reales dentro del test, e intenta que la relación entre los distintos datos sea evidente.

Otros patrones

- -Patrones de testing
- Patrones de diseño
- xUnitPatterns

Refactoring

- Forma disciplinada de introducir cambios reduciendo la posibilidad de introducir defectos (básicamente mejoro y reorganizo el código sin modificar el comportamiento ya validado)
- Es una transformación del software que
 - Preserva la estructura externa
 - Mejora la estructura interna
- Son cambios que se hacen en el sistema que
 - No cambien el comportamiento observable (todas las pruebas aún pasan)
 - Eliminan la duplicación o complejidad innecesaria
 - Mejoran la calidad del software
 - Hacen un código más fácil de entender y de cambiar
 - Flexibiliza el código
- Es necesario porque
 - Evita el "deterioro del diseño"
 - Incrementar la legibilidad y la comprensibilidad (ya que lo ordena y simplifica)

- Encuentra errores
- Reduce el tiempo de depuración
- Incorpora el aprendizaje que hacemos sobre la aplicación
- Rehacer las cosas es fundamental en todo proceso creativo

Como hacer refactoring



- El acto de diseñar test es uno de los mecanismos conocidos más efectivos para prevenir errores...El proceso mental que debe desarrollarse para crear test útiles puede descubrir y eliminar problemas en todas las etapas del desarrollo”